UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática

TFG del Grado en Ingeniería Informática

título del TFG
Documentación Técnica

Presentado por nombre alumno
en Universidad de Burgos — 15 de mayo de 2026
Tutor: nombre tutor

Índice general

Índice general **i**

Índice de figuras **iii**

Índice de tablas **iv**

Apéndice A Plan de Proyecto Software **1**
A.1. Introducción 1
A.2. Planificación temporal 2
A.3. Estudio de viabilidad 8

Apéndice B Especificación de Requisitos **11**
B.1. Introducción 11
B.2. Objetivos generales 11
B.3. Catálogo de requisitos 12
B.4. Especificación de requisitos 16

Apéndice C Especificación de diseño **31**
C.1. Introducción 31
C.2. Diseño de datos 31
C.3. Diseño arquitectónico 34
C.4. Diseño procedimental 36

Apéndice D Documentación técnica de programación **49**
D.1. Introducción 49
D.2. Estructura de directorios 50
D.3. Manual del programador 51

D.4. Compilación, instalación y ejecución del proyecto 54
D.5. Pruebas del sistema 57

Apéndice E Documentación de usuario 61
E.1. Introducción 61
E.2. Requisitos de usuarios 61
E.3. Instalación 63
E.4. Manual del usuario 65

Apéndice F Anexo de sostenibilización curricular 77
F.1. Introducción 77

Bibliografía 83

Índice de figuras

B.1. Diagrama de casos de uso del paciente. 20
B.2. Diagrama de casos de uso del administrador. 25

C.1. Relaciones lógicas entre entidades del sistema 33
C.2. Diagrama arquitectónico de módulos y dependencias 34
C.3. Arquitectura de despliegue del prototipo 36
C.4. Diagrama de secuencia - Triaje con confianza suficiente 38
C.5. Diagrama de secuencia - Ampliación de síntomas 40
C.6. Diagrama de secuencia - Fallback y selección manual 42
C.7. Diagrama de secuencia - Añadir síntoma desde administración . 44
C.8. Diagrama de secuencia - Revisión del log y promoción al CSV de revisados 45
C.9. Diagrama de secuencia - Regeneración de los modelos 46
C.10. Diagrama de secuencia - Acceso al panel de administración . . . 47

E.1. Barra lateral con la sesión de administrador activa 66
E.2. Pantalla principal del paciente 67
E.3. Ampliación de síntomas tras una predicción de baja confianza . 69
E.4. Selección manual de especialidad tras agotar las rondas 70
E.5. Configuración del umbral de confianza y de las rondas de ampliación 72
E.6. Formulario para añadir nuevos pares de síntomas y especialidad 73
E.7. Botones para regenerar todos los modelos o uno solo 74
E.8. Revisión y reasignación manual de casos de baja confianza o reportados como erróneos 75

Índice de tablas

A.1. Resumen de Sprints del proyecto 2
A.2. Coste de personal 8
A.3. Coste de recursos hardware (amortizado a 4 meses) 9
A.4. Coste total del proyecto 9
A.5. Licencias de las tecnologías utilizadas 10

B.1. A01 - Paciente. 17
B.2. A02 - Administrador. 18
B.3. A03 - Motor de clasificación. 19
B.4. A04 - Almacenamiento local. 20
B.5. CU-01 Introducir síntomas. 21
B.6. CU-02 Obtener especialidad recomendada. 22
B.7. CU-03 Resolver caso de baja confianza. 23
B.8. CU-04 Reportar error sobre predicción aceptada. 24
B.9. CU-05 Acceder a la pestaña de administración. 25
B.10. CU-06 Configurar el flujo de triaje. 26
B.11. CU-07 Añadir síntomas y especialidad. 27
B.12. CU-08 Revisar y resolver casos del log. 28
B.13. CU-09 Regenerar modelos. 29

Apéndice A

Plan de Proyecto Software

A.1. Introducción

La gestión de este proyecto software ha seguido una metodología ágil, organizando el trabajo en *Sprints* que permiten un desarrollo incremental y adaptativo. Dado que el proyecto se desarrolla de forma individual como Trabajo de Fin de Grado, la metodología adoptada es una variante simplificada de *Scrum*: se mantienen los Sprints como unidad de planificación y entrega, pero se prescinde de roles adicionales y de ceremonias formales como *daily standups*.

El seguimiento de tareas se ha realizado mediante un tablero Kanban gestionado a través de la plataforma *Zube.io* [9], integrada con el repositorio de GitHub. Las tarjetas se organizan en los estados propios de Zube.io: *Inbox* (entrada por defecto), *Backlog*, *Ready*, *In Progress*, *In Review* y *Done*, permitiendo un control visual del avance de cada Sprint. No se han utilizado Epics; en su lugar, los Sprints agrupan directamente las tareas relacionadas con cada componente del sistema.

El proyecto se articula en torno a cuatro bloques técnicos de modelado —*Term Frequency – Inverse Document Frequency* (TF-IDF) como línea base, TF-IDF con reducción *Singular Value Decomposition* (SVD), un clasificador *Support Vector Machine* (SVM) y una Red Neuronal Artificial (RNA) de tipo *Multi-layer Perceptron* (MLP)— a los que se añaden las fases de preparación de datos, desarrollo del frontal de usuario, panel de administración y documentación. La memoria y los anexos se han elaborado de forma paralela al desarrollo.

Sprint	Inicio	Fin
Introducción y Objetivos	19/02/2026	03/03/2026
Manejo y Edición con TeXTStudio	19/02/2026	03/03/2026
Búsqueda y conversión de DataSheets con datos	26/02/2026	23/04/2026
Creación de Script TF-IDF	01/04/2026	23/04/2026
Creación de Front End	21/04/2026	23/04/2026
Creación de Script TFIDF-SVD	22/04/2026	06/05/2026
Creación de Script SVM	26/04/2026	06/05/2026
Teoría del TFG — Memoria	03/03/2026	06/06/2026
Trabajos sobre el Front END	22/04/2026	01/06/2026
Adaptaciones de Código y Nuevas Funcionalidades	05/05/2026	01/06/2026
Anexos del TFG	06/05/2026	01/06/2026

Tabla A.1: Resumen de Sprints del proyecto

A.2. Planificación temporal

Visión general de los Sprints

A continuación se recoge la relación completa de Sprints realizados, con sus fechas de inicio y fin:

Detalle de los Sprints

Sprint 1 — Introducción y Objetivos

Periodo: 19 de febrero de 2026 – 3 de marzo de 2026.

Sprint inaugural dedicado a establecer la base documental del proyecto. Se definió la estructura de la memoria en LaTeX y se redactó el capítulo de introducción —incluyendo el análisis de la crisis de accesibilidad sanitaria, el estado del arte y la justificación tecnológica— así como la formulación de los objetivos generales, funcionales y no funcionales. El cierre se produjo tres días después de la fecha planificada, motivado por la curva de aprendizaje inicial con el entorno de documentación.

Sprint 2 — Manejo y Edición con TeXTStudio

Periodo: 19 de febrero de 2026 – 3 de marzo de 2026.

Sprint paralelo al primero, dedicado a la adquisición de competencia en TeXTStudio como herramienta principal de documentación técnica. Se trabajó en el manejo de abreviaturas y glosarios, la gestión de referencias bibliográficas con BibTEX, la inserción de figuras y tablas, y la compilación del documento completo. El cierre se adelantó dos días respecto a la fecha planificada.

Sprint 3 — Búsqueda y Conversión de DataSheets con Datos

Periodo: 26 de febrero de 2026 – 23 de abril de 2026.

Sprint dedicado a la localización, evaluación y transformación del conjunto de datos clínicos, en línea con el objetivo funcional de **preparación de los datos**. Los datos de partida consisten en un CSV con tres columnas: descripción de síntomas, enfermedad asociada y especialidad médica. Las tareas realizadas fueron:

- *Búsqueda de Datasheets:* exploración y evaluación de fuentes de datos clínicos públicas con licencia adecuada para el entrenamiento del clasificador.
- *Formato y Traducción:* normalización del formato de los datos, traducción y homogeneización de etiquetas de categorías médicas.
- *Aclaración de fuentes:* documentación de las fuentes seleccionadas, criterios de inclusión/exclusión y características del conjunto de datos definitivo.

Este sprint supuso la mayor desviación temporal del proyecto, cerrándose con casi siete semanas de retraso sobre lo planificado, debido a la escasez de *datasets* clínicos en español con las características requeridas.

Sprint 4 — Creación de Script TF-IDF

Periodo: 1 de abril de 2026 – 23 de abril de 2026.

Sprint centrado en el desarrollo del primer clasificador, que actúa como línea base dentro del objetivo funcional de **clasificación con cuatro modelos**. Se utilizó scikit-learn [5] como librería principal. Las tareas realizadas fueron:

- *Preparación del Entorno:* configuración del entorno de desarrollo Python con las dependencias necesarias (scikit-learn, pandas, numpy).

- *Pruebas en Google Colab:* validación inicial del *pipeline* de Procesamiento de Lenguaje Natural (PLN) y del vectorizador TF-IDF en un entorno de experimentación rápida.
- *Programación:* implementación completa del *pipeline* de preprocesado (normalización, eliminación de *stopwords*, tokenización y generación de bigramas y trigramas) y del clasificador TF-IDF base.
- *A Python y Streamlit tf_idf:* integración del script de clasificación TF-IDF con el entorno de ejecución local y primeras pruebas con Streamlit [8].
- *Separar el módulo que genera el modelo:* refactorización del código para desacoplar la generación y persistencia del modelo de la lógica de predicción, en línea con el objetivo no funcional de **modularidad**.

El sprint se cerró un día antes de lo planificado.

Sprint 5 — Creación de Front End

Periodo: 21 de abril de 2026 – 23 de abril de 2026.

Sprint dedicado al desarrollo de la interfaz de usuario con Streamlit [8], en línea con el objetivo funcional de **predicción a partir de texto libre**:

- *Creación de app.py:* desarrollo del frontal que permite al paciente introducir síntomas por escrito y recibir la especialidad médica predicha junto con el porcentaje de confianza. Se incorporó también la lógica de umbral: cuando la confianza queda por debajo del valor configurado, el sistema solicita al paciente que amplíe la descripción o elige entre las alternativas más probables.

El sprint se cerró con cinco días de antelación respecto a la fecha planificada.

Sprint 6 — Creación de Script TFIDF-SVD

Periodo: 22 de abril de 2026 – 6 de mayo de 2026.

Sprint dedicado al segundo clasificador del objetivo funcional de **clasificación con cuatro modelos**: TF-IDF combinado con reducción de dimensionalidad mediante TruncatedSVD, implementado con scikit-learn [5]. Las tareas realizadas fueron:

- *Separar generar Módulo:* desacoplamiento del módulo de generación del modelo SVD para garantizar la persistencia de transformaciones y la consistencia entre entrenamiento y predicción.

- *Implementación en el app.py:* integración del modelo TF-IDF+SVD en el frontal Streamlit, permitiendo seleccionar el modelo de clasificación activo.

- *Creación del Script:* implementación de TruncatedSVD sobre la matriz TF-IDF, selección del número de componentes mediante validación cruzada y evaluación del rendimiento (F1 macro/micro, tiempo de entrenamiento y consumo de memoria) frente al modelo base.

El sprint se cerró con una semana de retraso sobre lo planificado, debido al tiempo requerido para el análisis sistemático de la varianza explicada en función del número de componentes.

Sprint 7 — Creación de Script SVM

Periodo: 26 de abril de 2026 – 6 de mayo de 2026.

Sprint dedicado al tercer clasificador del objetivo funcional de **clasificación con cuatro modelos**: un SVM lineal entrenado con scikit-learn [5]. Las tareas realizadas fueron:

- *Creación:* implementación del clasificador SVM sobre las representaciones TF-IDF y TF-IDF+SVD, con búsqueda de hiperparámetros (`C`, `class_weight`) mediante `GridSearchCV` y validación cruzada estratificada optimizando macro-F1.

- *Implementación en el Front End:* integración del modelo SVM en el frontal Streamlit como opción de clasificación seleccionable.

- *Separar Generar Modelo:* refactorización para desacoplar la generación y persistencia del modelo SVM del módulo de predicción.

El sprint se cerró cuatro días antes de la fecha planificada.

Sprint 8 — Teoría del TFG — Memoria

Periodo: 3 de marzo de 2026 – 6 de junio de 2026.

Sprint de larga duración, planificado en paralelo con el desarrollo, que cubre la redacción del cuerpo teórico de la memoria. Las tareas contempladas son:

- *Búsqueda de Teoría en la red:* recopilación y análisis de fuentes bibliográficas sobre PLN, clasificación de texto clínico, TF-IDF, SVD, SVM y MLP.
- *Conceptos Teóricos:* redacción del capítulo de fundamentos teóricos de la memoria.
- *Añadir Referencias Teoría:* incorporación y revisión de las referencias bibliográficas al documento.
- *Convertir a LaTeX:* conversión y maquetación del contenido teórico al formato LaTeX de la plantilla del TFG.
- *Técnicas y Herramientas:* redacción del capítulo de técnicas y herramientas utilizadas.
- *Aspectos relevantes del desarrollo del proyecto:* redacción del capítulo de aspectos relevantes del desarrollo.
- *Trabajos Relacionados:* análisis y redacción del capítulo de trabajos relacionados y estado del arte.
- *Conclusiones y Líneas de trabajo Futuras:* redacción del capítulo de conclusiones y trabajo futuro.

Sprint 9 — Trabajos sobre el Front END

Periodo: 22 de abril de 2026 – 1 de junio de 2026.

Sprint que extiende el frontal Streamlit [8] con la integración progresiva de los modelos y la mejora de la experiencia de usuario. Las tareas contempladas son:

- *Inclusión TFIDF:* integración del modelo TF-IDF en la interfaz como opción de clasificación seleccionable.
- *Inclusión SVD:* integración del modelo TF-IDF+SVD en la interfaz.
- *Inclusión SVM en Front End:* integración del clasificador SVM en la interfaz.

- *Añadir Síntomas - Especialidad en Front End:* mejora de la interfaz para mostrar la especialidad médica asociada a cada predicción, incluyendo el indicador de confianza y la lógica de segunda vuelta cuando la predicción queda por debajo del umbral configurado.

Sprint 10 — Adaptaciones de Código y Nuevas Funcionalidades

Periodo: 5 de mayo de 2026 – 1 de junio de 2026.

Sprint que incorpora el cuarto clasificador y el panel de administración, completando así los objetivos funcionales de **clasificación con cuatro modelos** y **panel de administración**. Las tareas contempladas son:

- *Creación del Módulo Red Neuronal:* implementación del cuarto clasificador, una RNA de tipo MLP entrenada con *early stopping* y regularización para evitar sobreajuste. Este modelo completa el conjunto de cuatro enfoques comparados bajo el mismo protocolo de evaluación.
- *Creación Módulo Administrador:* desarrollo del panel de administración con tres herramientas: añadir nuevos pares síntoma-especialidad a un CSV ampliado, revisar y etiquetar el log de consultas no clasificadas con confianza suficiente, y disparar el reentrenamiento de los cuatro modelos con los datos actualizados.
- *Securizar el Módulo Administrador:* implementación de mecanismos de autenticación y control de acceso al panel de administración para restringir su uso al perfil autorizado.

Sprint 11 — Anexos del TFG

Periodo: 6 de mayo de 2026 – 1 de junio de 2026.

Sprint dedicado íntegramente a la redacción de los anexos de la memoria. Las tareas contempladas son:

- *Plan de Proyecto:* redacción del presente apéndice A.
- *Especificación de Requisitos:* redacción del apéndice B con el catálogo de requisitos funcionales y no funcionales.
- *Diseño:* redacción del apéndice C con la especificación de diseño arquitectónico y de datos.

- *Manual del Programador:* redacción del apéndice D con la documentación técnica de programación.
- *Manual del Usuario:* redacción del apéndice E con la guía de instalación y uso de la aplicación.
- *Anexo de sostenibilización curricular:* redacción del apéndice F con la reflexión sobre sostenibilidad.

A.3. Estudio de viabilidad

Viabilidad económica

El proyecto se ha desarrollado íntegramente sobre tecnologías de software libre y código abierto, por lo que el coste de licencias es nulo. A continuación se simula el coste real que supondría el desarrollo en un contexto empresarial.

Costes de personal

Se simula la contratación de un ingeniero informático junior durante los aproximadamente 4 meses de desarrollo efectivo (febrero–mayo de 2026). El salario de referencia se ha obtenido del XVIII Convenio Colectivo Estatal de Empresas de Consultoría y Tecnologías de la Información [3], y los tipos de cotización corresponden a los publicados por la Seguridad Social [6] y la Agencia Tributaria [1] para el ejercicio 2026.

Concepto	**Importe**
Salario mensual bruto	1.800,00 €
Retención IRPF (13,55 %)	243,90 €
Cotización SS empresa (36,25 %)	652,50 €
Coste mensual para la empresa	**2.452,50 €**
Total 4 meses	**9.810,00 €**

Tabla A.2: Coste de personal

Costes de recursos hardware

Se considera únicamente el equipo de trabajo del desarrollador (trabajo desde domicilio), calculando el coste amortizado para los 4 meses de duración del proyecto.

Concepto	Precio sin IVA	Coef.	Coste 4 meses
Ordenador portátil	750,00 €	26 %	65,00 €
Monitor adicional	160,00 €	30 %	16,00 €
Teclado y ratón	30,00 €	30 %	3,00 €
Total	**940,00 €**	—	**84,00 €**

Tabla A.3: Coste de recursos hardware (amortizado a 4 meses)

Coste total del proyecto

Concepto	Coste
Coste de personal (4 meses)	9.810,00 €
Coste de hardware (amortizado)	84,00 €
Herramientas software	0,00 €
TOTAL	**9.894,00 €**

Tabla A.4: Coste total del proyecto

Beneficios y modelo de explotación

Aunque el prototipo se desarrolla como TFG sin ánimo de lucro inmediato, se identifican dos vías de explotación potencial en un contexto empresarial:

- **Servicio SaaS para centros de salud:** la aplicación se ofrecería como servicio alojado con tarifas por volumen de consultas clasificadas (p. ej., cuota mensual por centro).
- **Solución standalone con soporte:** distribución del sistema empaquetado (p. ej., contenedor Docker) con diferentes niveles de soporte técnico y actualización de modelos con los datos propios de cada centro.

Viabilidad legal

El proyecto se apoya exclusivamente en librerías de código abierto con licencias compatibles entre sí y con el uso académico y comercial. Además, al ejecutarse en local sin enviar datos a servicios externos, el sistema es

compatible por diseño con los principios de minimización y soberanía del dato del **Reglamento General de Protección de Datos (RGPD)**.

Tecnología	**Licencia**
Python 3.11	PSF License (permisiva, uso comercial permitido)
scikit-learn, numpy, pandas [5]	BSD 3-Clause (permisiva, uso comercial permitido)
Streamlit [8]	Apache License 2.0 (permisiva, uso comercial permitido)
LaTeX / TeXTStudio	LPPL / GPL (libre para uso académico y personal)
Zube.io [9]	Propietaria, plan gratuito para proyectos individuales
Git	GPL v2 (libre)

Tabla A.5: Licencias de las tecnologías utilizadas

La licencia más restrictiva del conjunto es la GPL, presente solo en herramientas de desarrollo que no forman parte del artefacto distribuible. El código fuente de la aplicación puede distribuirse bajo licencia **MIT**, al no incorporar componentes GPL en el producto final.

En cuanto a los datos, el conjunto utilizado para el entrenamiento procede de fuentes públicas con licencias que permiten su uso en investigación y proyectos académicos. Para un despliegue en producción con datos reales de pacientes, sería necesario cumplir adicionalmente con la **Ley Orgánica de Protección de Datos (LOPD-GDD)**, garantizando el consentimiento informado y la anonimización de los registros clínicos.

Apéndice B

Especificación de Requisitos

B.1. Introducción

Este anexo recoge la especificación de requisitos del prototipo standalone desarrollado para clasificar síntomas en lenguaje natural y asignar la especialidad médica correspondiente. La aplicación se ejecuta en local, sin dependencias de servicios externos, y aborda el problema del triaje y la asignación de citas planteado en la introducción de la memoria.

El documento sirve como referencia funcional durante el análisis y el diseño, y también como base para comprobar el cumplimiento de los requisitos en las fases de validación y pruebas.

La aplicación tiene dos pestañas. La del **paciente** permite escribir los síntomas en texto libre y devuelve la especialidad predicha junto con un indicador de confianza. La del **administrador**, protegida por contraseña, sirve para gestionar el catálogo de síntomas, ajustar la configuración del triaje, revisar los casos no resueltos y reentrenar los cuatro modelos: TF-IDF, TF-IDF con reducción mediante SVD, SVM y una RNA de tipo MLP.

B.2. Objetivos generales

Los objetivos generales del proyecto se recogen en el capítulo 2 de la memoria y son los que han guiado los requisitos de este anexo:

- **Clasificación automática de síntomas en lenguaje natural:** a partir de la descripción libre del paciente, devolver la especialidad médica más adecuada y un indicador de confianza.

- **Flujo completo de PLN:** cubrir en un único ejecutable la validación de datos, el preprocesado y la predicción.
- **Comparativa de cuatro modelos:** entrenar y evaluar TF-IDF como línea base, TF-IDF con reducción mediante SVD, SVM y un Perceptrón Multicapa, midiendo el acierto y también el coste en tiempo y memoria.
- **Estrategia multimodelo en producción:** ejecutar los cuatro modelos en paralelo y elegir automáticamente el de mayor confianza, con la opción de forzar uno concreto.
- **Gestión administrativa:** el administrador puede ampliar el catálogo de datos con nuevos pares síntoma–especialidad, ajustar el umbral de confianza y los intentos de reintento, revisar los casos pendientes en el *log* de fallback y ejecutar el reentrenamiento de los modelos.
- **Ejecución local y privacidad por diseño:** todo (datos, modelos y predicción) se ejecuta en el equipo del usuario. Sin servicios cloud y sin modelos de IA propietarios de terceros.
- **Modularidad:** arquitectura por módulos para facilitar el mantenimiento y poder añadir más adelante nuevos canales de entrada o modelos.

B.3. Catálogo de requisitos

Los requisitos se agrupan en dos bloques: funcionales y no funcionales.

Requisitos funcionales

- **RF-1 Gestión de datos:** la aplicación debe gestionar el conjunto de datos de entrenamiento de forma persistente.
  - **RF-1.1 Carga del dataset principal:** el sistema debe cargar un CSV de partida con cuatro columnas (identificador de registro, descripción de síntomas, enfermedad asociada y especialidad médica), codificado en Latin-1 y separado por punto y coma.
  - **RF-1.2 Validación del dataset:** antes del entrenamiento, el sistema descarta valores nulos, líneas malformadas y registros incompletos.

  - **RF-1.3 Mantenimiento de un CSV de revisados:** las aportaciones generadas a partir del *log* de baja confianza se almacenan en un CSV adicional, independiente del dataset principal.
  - **RF-1.4 Incorporación directa al dataset principal:** el administrador debe poder añadir nuevos pares síntoma–especialidad directamente al dataset principal con un identificador correlativo único.
  - **RF-1.5 Combinación de fuentes para entrenamiento:** en el reentrenamiento, el sistema debe combinar el dataset principal y el CSV de revisados como conjunto de datos único.

- **RF-2 Preprocesado del texto:** el sistema aplica el mismo pipeline de PLN en entrenamiento y en predicción.
  - **RF-2.1 Limpieza y normalización:** el sistema debe convertir el texto a minúsculas, eliminar caracteres no alfabéticos preservando tildes y la letra ñ, y tokenizar el resultado.
  - **RF-2.2 Eliminación de palabras vacías:** se deben eliminar las *stopwords* del español antes de la vectorización.
  - **RF-2.3 Reducción morfológica (Stemming):** se debe aplicar el algoritmo Snowball para español sobre los *tokens* resultantes.
  - **RF-2.4 Generación de n-gramas:** la vectorización debe contemplar al menos unigramas y bigramas para captar expresiones clínicas compuestas como “dolor torácico” o “falta de aire”.
  - **RF-2.5 Coherencia entrenamiento/predicción:** el preprocesado aplicado al texto introducido por el paciente debe ser el mismo que el utilizado durante el entrenamiento.

- **RF-3 Modelos de clasificación:** la aplicación integra los cuatro modelos del proyecto, con una interfaz común para entrenarlos, persistirlos y consultarlos.
  - **RF-3.1 Modelo TF-IDF:** el sistema debe disponer de un clasificador basado en representación TF-IDF combinado con regresión logística como línea base.
  - **RF-3.2 Modelo TF-IDF + SVD:** el sistema debe disponer de un modelo que aplique reducción de dimensionalidad mediante TruncatedSVD sobre la representación TF-IDF antes de la clasificación.

- **RF-3.3 Modelo SVM:** el sistema debe disponer de un clasificador SVM con kernel lineal y `probability=True` entrenado sobre la representación TF-IDF.
- **RF-3.4 Modelo RNA (MLP):** el sistema debe disponer de un Perceptrón Multicapa entrenado con early stopping y regularización L2 (penalización proporcional al cuadrado de los pesos de la red) para evitar el sobreajuste.
- **RF-3.5 Persistencia de modelos:** los modelos entrenados deben serializarse en disco con `joblib` para evitar el reentrenamiento en cada arranque.
- **RF-3.6 Estimación de probabilidades:** cada modelo debe devolver, junto con la especialidad predicha, una puntuación de confianza expresada como porcentaje.

- **RF-4 Pestaña del paciente:** la aplicación debe ofrecer una pestaña destinada al paciente que permita la introducción de síntomas y la consulta de la especialidad asignada.
  - **RF-4.1 Entrada de síntomas por texto:** el paciente debe poder introducir sus síntomas escribiéndolos en un campo de texto libre.
  - **RF-4.2 Modo multimodelo (por defecto):** por defecto, el sistema debe ejecutar los cuatro modelos en paralelo y seleccionar automáticamente la predicción del modelo con mayor confianza.
  - **RF-4.3 Selección manual de modelo:** el paciente debe poder elegir alternativamente un modelo concreto entre TF-IDF, SVD, SVM o RNA.
  - **RF-4.4 Predicción de especialidad:** el sistema debe devolver la especialidad médica recomendada junto con la confianza asociada.
  - **RF-4.5 Visualización comparativa:** cuando se utilice el modo multimodelo, el sistema debe mostrar al paciente una tabla con los resultados de los cuatro modelos, identificando el modelo elegido.
  - **RF-4.6 Umbral de confianza configurable:** debe existir un umbral configurable por debajo del cual la predicción se considere dudosa.
  - **RF-4.7 Intentos extra configurables:** debe poderse configurar el número de intentos en los que se pide al paciente ampliar la descripción de los síntomas cuando la confianza es insuficiente.

- **RF-4.8 Solicitud de información adicional:** si la confianza queda por debajo del umbral y quedan intentos disponibles, el sistema debe pedir al paciente que amplíe la descripción y volver a evaluar la predicción.
- **RF-4.9 fallback por baja confianza:** si tras agotar los intentos la confianza sigue sin alcanzar el umbral, el sistema debe ofrecer al paciente dos opciones: aceptar la sugerencia del modelo o elegir manualmente otra especialidad de las disponibles en el dataset (con Medicina General como opción por defecto).
- **RF-4.10 Reporte de error sobre predicción aceptada:** aun cuando la confianza haya superado el umbral, el paciente debe poder marcar el resultado como incorrecto e indicar la especialidad correcta.
- **RF-4.11 Registro en log de baja confianza y reportes:** todos los casos resueltos por fallback y todos los reportes de error deben quedar registrados en un *log*, indicando síntomas y especialidad sugerida por el modelo o elegida por el paciente.

- **RF-5 Pestaña del administrador:** la aplicación debe ofrecer una pestaña destinada al administrador con las herramientas necesarias para mantener el sistema.
  - **RF-5.1 Autenticación del administrador:** el acceso a la pestaña debe requerir usuario y contraseña; la contraseña debe poder configurarse mediante variable de entorno o fichero de secretos, sin quedar incrustada en el código.
  - **RF-5.2 Configuración del triaje:** el administrador debe poder modificar el umbral mínimo de confianza y el número de intentos extra, persistiendo los cambios en un fichero de configuración (`config.json`).
  - **RF-5.3 Añadir síntomas y especialidad al dataset principal:** el administrador debe poder incorporar nuevas entradas (síntomas, enfermedad opcional, especialidad) al dataset principal.
  - **RF-5.4 Selección de especialidad existente o nueva:** la incorporación de nuevos registros debe permitir seleccionar una especialidad ya presente en el dataset o introducir una nueva.
  - **RF-5.5 Visualización del log:** el administrador debe poder consultar el listado de casos registrados en el *log* de baja confianza y reportes de error.

- **RF-5.6 Resolución de casos del log:** el administrador debe poder asignar manualmente la especialidad correcta a cada caso del *log* y volcarlo al CSV de revisados.
- **RF-5.7 Regeneración conjunta o individual de modelos:** el administrador debe poder regenerar los cuatro modelos a la vez a partir del dataset actualizado mediante una única acción o regenerarlos por separado.

Requisitos no funcionales

- **RNF-1 Ejecución standalone:** la aplicación debe ejecutarse de forma totalmente local, sin depender de APIs externas, servicios en la nube ni conexión a internet para la operativa principal.
- **RNF-2 Sin dependencia de IA propiedad de terceros:** el sistema no debe apoyarse en LLMs comerciales ni en servicios externos de clasificación clínica.
- **RNF-3 Privacidad por diseño:** los datos clínicos manipulados (síntomas, log de incidencias, dataset) deben permanecer en el equipo en el que se ejecuta la aplicación.
- **RNF-4 Ejecución sobre CPU:** los modelos deben ser lo bastante ligeros como para entrenarse y ejecutarse en un equipo modesto.
- **RNF-5 Rendimiento razonable:** los tiempos de respuesta de predicción y reentrenamiento deben de completarse en un plazo asumible.
- **RNF-6 Usabilidad:** la interfaz debe ser intuitiva, con una curva de aprendizaje baja.
- **RNF-7 Modularidad:** el código debe estar separado en módulos con responsabilidades bien delimitadas (carga de datos, preprocesado, vectorización, modelos, evaluación e interfaz).
- **RNF-8 Portabilidad:** el sistema, desarrollado en Python con dependencias acotadas en `requirements.txt`, debe poder ejecutarse en distintos sistemas operativos sin modificaciones sustanciales.

B.4. Especificación de requisitos

Esta sección recoge los actores del sistema, el diagrama general de casos de uso y la descripción de cada caso.

Actores del sistema

A01	**Paciente**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Descripción	Persona que utiliza la aplicación para describir sus síntomas y obtener una recomendación de especialidad médica.
Tipo	Usuario
Objetivo	Obtener una clasificación fiable de su sintomatología en una especialidad médica para orientar la solicitud de cita.
Responsabilidades	▪ Introducir sus síntomas en texto libre. ▪ Elegir el modo de clasificación (multimodelo o modelo concreto). ▪ Ampliar la descripción o elegir entre las alternativas propuestas si el sistema lo solicita. ▪ Reportar como incorrecta una predicción si lo considera oportuno.
Relaciones con casos de uso	CU-01, CU-02, CU-03, CU-04

Tabla B.1: A01 - Paciente.

A02	**Administrador**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Descripción	Usuario con privilegios que mantiene el catálogo de datos, la configuración del triaje y los modelos. Necesita autenticarse para acceder a la pestaña restringida.
Tipo	Usuario
Objetivo	Mantener actualizados los datos y los modelos, y ajustar el flujo de triaje cuando sea necesario.
Responsabilidades	▪ Configurar el umbral de confianza y los intentos extra. ▪ Añadir nuevos pares síntoma–especialidad al dataset principal. ▪ Revisar el *log* y resolver casos pendientes. ▪ Regenerar los modelos, de forma conjunta o individual.
Relaciones con casos de uso	CU-05, CU-06, CU-07, CU-08, CU-09

Tabla B.2: A02 - Administrador.

A03	**Motor de clasificación**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Descripción	Módulo interno que aplica el preprocesado y los cuatro modelos (TF-IDF, TF-IDF+SVD, SVM y RNA) sobre el texto introducido, y devuelve la especialidad predicha con su confianza. Puede usarse en modo individual o en modo multimodelo.
Tipo	Sistema
Objetivo	Clasificar texto libre en una especialidad médica y devolver una confianza asociada.
Responsabilidades	▪ Preprocesar el texto introducido. ▪ Aplicar uno o varios modelos de clasificación. ▪ Calcular el indicador de confianza por modelo. ▪ Seleccionar el modelo con mayor confianza en modo multimodelo.
Relaciones con casos de uso	CU-02, CU-03, CU-09

Tabla B.3: A03 - Motor de clasificación.

A04	**Almacenamiento local**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Descripción	Persistencia local del sistema: dataset principal y CSV de revisados, *log* de baja confianza y reportes de error, modelos serializados con `joblib` y fichero de configuración `config.json`.
Tipo	Sistema
Objetivo	Conservar datos, modelos y configuración en local, sin recurrir a servicios externos.
Responsabilidades	▪ Almacenar el dataset principal y el CSV de revisados. ▪ Registrar los casos en el *log*. ▪ Persistir los modelos entrenados y la configuración del triaje.
Relaciones con casos de uso	CU-03, CU-04, CU-05, CU-06, CU-07, CU-08, CU-09

Tabla B.4: A04 - Almacenamiento local.

Diagrama general de casos de uso

A continuación se muestran dos diagramas separados para mejorar la legibilidad: uno con los casos de uso del paciente y otro con los del administrador.

Casos de uso

CU-01	**Introducir síntomas**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A01 Paciente
Requisitos asociados	RF-4.1, RF-4.2, RF-4.3
Descripción	El paciente introduce sus síntomas en el formulario y selecciona el modo de clasificación (multimodelo o modelo concreto).
Precondición	La aplicación está abierta en la pestaña del paciente.
Acciones	1. El paciente accede a la pestaña Triaje. 2. Escribe sus síntomas en el campo de texto. 3. Selecciona el modelo (Todos por defecto, o uno concreto). 4. Pulsa el botón “Realizar triaje”.
Postcondición	El sistema dispone del texto y la configuración necesaria para iniciar la clasificación.
Excepciones	Campo de síntomas vacío al pulsar el botón.
Importancia	Alta

Tabla B.5: CU-01 Introducir síntomas.

CU-02	**Obtener especialidad recomendada**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A01 Paciente, A03 Motor de clasificación, A04 Almacenamiento local
Requisitos asociados	RF-2, RF-3, RF-4.2, RF-4.3, RF-4.4, RF-4.5
Descripción	El sistema procesa los síntomas y devuelve la especialidad recomendada y su porcentaje de confianza. En modo multimodelo, muestra además los resultados de los cuatro modelos.
Precondición	Existen modelos entrenados y persistidos y el paciente ha introducido síntomas.
Acciones	1. El sistema aplica el preprocesado sobre el texto introducido. 2. Ejecuta los cuatro modelos (modo multimodelo) o el modelo seleccionado. 3. En modo multimodelo, escoge la predicción con mayor confianza. 4. Muestra al paciente la especialidad recomendada y la confianza; en modo multimodelo, también la tabla comparativa.
Postcondición	El paciente conoce la especialidad recomendada y, en su caso, el desempeño de cada modelo.
Excepciones	Modelos no disponibles o corruptos; ningún modelo devuelve un resultado válido.
Importancia	Alta

Tabla B.6: CU-02 Obtener especialidad recomendada.

CU-03	**Resolver caso de baja confianza**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A01 Paciente, A03 Motor de clasificación, A04 Almacenamiento local
Requisitos asociados	RF-4.6, RF-4.7, RF-4.8, RF-4.9, RF-4.11
Descripción	Si la confianza está por debajo del umbral, el sistema pide más síntomas durante los intentos configurados. Si tras agotarlos la confianza sigue siendo insuficiente, el paciente puede aceptar la sugerencia del modelo o elegir manualmente otra especialidad. El caso queda registrado en el *log*.
Precondición	Se ha realizado una primera predicción cuya confianza es inferior al umbral configurado.
Acciones	1. El sistema solicita al paciente ampliar la descripción de los síntomas. 2. El paciente añade información complementaria. 3. El sistema reevalúa la predicción con el mismo modelo. 4. Si la confianza es suficiente, devuelve la especialidad y termina. 5. Si la confianza sigue siendo insuficiente y quedan intentos, se repite el paso 1. 6. Agotados los intentos, el sistema ofrece al paciente aceptar la sugerencia del modelo o elegir manualmente otra especialidad (Medicina General por defecto). 7. El caso queda registrado en el *log* con motivo `baja_confianza`.
Postcondición	El paciente recibe una recomendación final y el caso queda registrado en el *log*.
Excepciones	El paciente cancela el proceso antes de finalizar los intentos.
Importancia	Alta

Tabla B.7: CU-03 Resolver caso de baja confianza.

CU-04	**Reportar error sobre predicción aceptada**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A01 Paciente, A04 Almacenamiento local
Requisitos asociados	RF-4.10, RF-4.11
Descripción	Cuando la confianza ha superado el umbral, el paciente puede marcar la predicción como incorrecta e indicar la especialidad real.
Precondición	El paciente ha recibido una recomendación de especialidad con confianza por encima del umbral.
Acciones	1. El paciente pulsa el botón "Reportar error". 2. Selecciona del catálogo de especialidades la que considera correcta. 3. Confirma el envío del reporte. 4. El sistema registra el caso en el *log* con motivo `reporte_error`.
Postcondición	El caso queda almacenado en el *log* para su revisión posterior por el administrador.
Excepciones	Error de escritura en el *log*; cancelación por parte del paciente.
Importancia	Media

Tabla B.8: CU-04 Reportar error sobre predicción aceptada.

CU-05	**Acceder a la pestaña de administración**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A02 Administrador
Requisitos asociados	RF-5.1
Descripción	El administrador inicia sesión con usuario y contraseña para acceder a las funcionalidades restringidas.
Precondición	Existe una contraseña configurada en `st.secrets`, en variable de entorno o en fichero de secretos.
Acciones	1. El administrador selecciona la pestaña Administración. 2. Introduce usuario y contraseña. 3. Pulsa el botón "Entrar". 4. El sistema valida las credenciales mediante comparación segura.
Postcondición	El administrador queda autenticado y accede a las herramientas de gestión.
Excepciones	Credenciales incorrectas; no hay contraseña configurada.
Importancia	Alta

Tabla B.9: CU-05 Acceder a la pestaña de administración.

CU-06	**Configurar el flujo de triaje**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A02 Administrador, A04 Almacenamiento local
Requisitos asociados	RF-4.6, RF-4.7, RF-5.2
Descripción	El administrador ajusta el umbral de confianza y el número de intentos extra de ampliación.
Precondición	El administrador está autenticado.
Acciones	1. El administrador modifica el umbral mediante un *slider*. 2. Modifica el número de intentos extra. 3. Pulsa "Guardar configuración". 4. El sistema persiste los valores en `config.json`.
Postcondición	Las próximas predicciones utilizan la nueva configuración.
Excepciones	Error de escritura en disco.
Importancia	Media

Tabla B.10: CU-06 Configurar el flujo de triaje.

CU-07	**Añadir síntomas y especialidad**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A02 Administrador, A04 Almacenamiento local
Requisitos asociados	RF-1.4, RF-5.3, RF-5.4
Descripción	El administrador añade nuevos pares síntoma–especialidad al dataset principal, eligiendo entre especialidades existentes o introduciendo una nueva.
Precondición	El administrador está autenticado.
Acciones	1. El administrador introduce la descripción de los síntomas. 2. Selecciona una especialidad existente o escribe una nueva. 3. Si quiere, indica la enfermedad asociada. 4. Pulsa “Guardar registro”. 5. El sistema asigna un identificador correlativo y añade la fila al dataset principal.
Postcondición	El dataset principal contiene una entrada nueva, disponible para el próximo reentrenamiento.
Excepciones	Datos incompletos; error de escritura.
Importancia	Alta

Tabla B.11: CU-07 Añadir síntomas y especialidad.

CU-08	**Revisar y resolver casos del log**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A02 Administrador, A04 Almacenamiento local
Requisitos asociados	RF-1.3, RF-5.5, RF-5.6
Descripción	El administrador consulta los casos del *log* (baja confianza y reportes de error). Para los que decida revisar, asigna la especialidad correcta y vuelca el caso al CSV de revisados.
Precondición	Existe al menos un caso registrado en el *log*.
Acciones	1. El administrador accede al apartado de log. 2. Visualiza cada caso con los síntomas. 3. Para cada caso a revisar, selecciona la especialidad adecuada. 4. Pulsa "Añadir a datos" para incorporar el caso al CSV de revisados.
Postcondición	El caso queda incorporado al CSV de revisados; el log mantiene su registro histórico.
Excepciones	Especialidad no válida; fallo en la escritura del CSV.
Importancia	Alta

Tabla B.12: CU-08 Revisar y resolver casos del log.

CU-09	**Regenerar modelos**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A02 Administrador, A03 Motor de clasificación, A04 Almacenamiento local
Requisitos asociados	RF-1.5, RF-3.5, RF-5.7
Descripción	El administrador reentrena los modelos a partir del dataset principal combinado con el CSV de revisados. Puede hacerlo de forma conjunta o regenerar un solo modelo.
Precondición	Existen los CSV de datos accesibles y los scripts de regeneración.
Acciones	1. El administrador pulsa "Regenerar todos los modelos" o un botón individual. 2. El sistema invoca el script correspondiente a cada modelo y muestra un indicador de progreso. 3. Cada script combina el dataset principal y el CSV de revisados. 4. El script entrena el modelo y persiste el resultado en disco. 5. El sistema notifica al administrador el resultado de cada regeneración.
Postcondición	Los modelos persistidos reflejan los datos actualizados y las aportaciones del administrador.
Excepciones	Datos corruptos; error durante el entrenamiento; fallo de escritura; script no encontrado.
Importancia	Alta

Tabla B.13: CU-09 Regenerar modelos.

Apéndice C

Especificación de diseño

C.1. Introducción

Este anexo recoge las decisiones de diseño del prototipo de triaje clínico desarrollado en el TFG, organizadas en tres planos. El diseño de datos describe qué información maneja la aplicación y cómo se guarda en el sistema de ficheros. El diseño arquitectónico explica cómo se reparte el software en módulos y cómo `app.py` orquesta al resto. Y el diseño procedimental, ya con diagramas de secuencia, sigue los casos de uso del anexo B paso a paso.

C.2. Diseño de datos

El prototipo no usa una base de datos.Trabaja con varios ficheros planos. Algunos se cargan en memoria al arrancar y otros se leen solo cuando se necesitan. Para un conjunto de unos pocos miles de filas, que apenas varía y que utiliza un único usuario, los CSV cumplen sin problema y resultan más cómodos de versionar y revisar que un motor SQL.

Entidades del sistema

No hay esquema relacional formal, pero sí cinco entidades lógicas que estructuran la información:

- **RegistroSintoma**: una fila del dataset principal. Lleva el identificador (*record_id*), el texto libre con los síntomas (*symptoms_text*), la enfermedad asociada cuando se conoce (*disease*, opcional) y la especialidad médica de destino (*specialty*), que es la clase objetivo del clasificador.

- **PrediccionTriaje**: el resultado de una consulta del paciente. Vive solo en memoria. Lleva la *especialidad* devuelta, la *confianza* asociada y el *modelo_origen* que la produjo (TF-IDF, SVD, SVM o RNA). Cuando se ejecuta el modo *Todos*, la aplicación construye un diccionario con una predicción por modelo y se queda con la de mayor confianza.

- **EntradaLog**: una fila del log de fallback. Se persiste en

  `triaje_fallback_log` cuando el sistema agota las rondas de ampliación sin alcanzar el umbral, o cuando el paciente reporta que la sugerencia recibida no era correcta. Tiene nueve campos: marca temporal, clasificador empleado, rondas realizadas, umbral aplicado, síntomas registrados, especialidad sugerida por el modelo, confianza del modelo, especialidad finalmente elegida por el paciente y motivo (`baja_confianza` o `reporte_error`).

- **RegistroRevisado**: una fila promovida desde el log al CSV de revisados (`Datos/triajes_baja_confianza_revisados.csv`). Solo guarda dos campos: *symptoms_text* y *specialty*, los mínimos para que los scripts de regeneración puedan usarla como ejemplo extra de entrenamiento.

- **Configuracion**: dos parámetros que controlan el comportamiento del triaje y que pueden cambiarse desde el panel de administración. El *umbral_confianza* marca a partir de qué porcentaje la predicción se considera buena (45 % por defecto). Las *rondas_extra* fijan cuántas veces se pedirá al paciente ampliar los síntomas antes de pasar al fallback (1 por defecto). Se guarda en `config.json`.

- **ModeloEntrenado**: representa un clasificador ya entrenado y guardado en disco, listo para volver a usarse sin tener que entrenarlo de nuevo. Incluye todas las piezas necesarias para predecir: el vectorizador, el reductor de dimensionalidad si lo utiliza, el clasificador y la codificación de etiquetas. Toda esa información se guarda en uno o varios ficheros `.joblib` dentro de la carpeta del modelo, y se recupera con la misma librería `joblib` en el momento de hacer una predicción.

Relaciones entre entidades

La figura C.1 resume las relaciones entre las entidades anteriores. El dataset base y el CSV de revisados alimentan, juntos, el entrenamiento de los cuatro modelos. Cada modelo produce sus propios ficheros `.joblib`. El paciente genera predicciones que normalmente se quedan en memoria, salvo cuando entran en el log de fallback. Y, desde el panel de administración, las

entradas del log pueden promoverse al CSV de revisados, cerrando el ciclo: lo que un paciente no consiguió clasificar bien hoy puede acabar mejorando los modelos mañana.

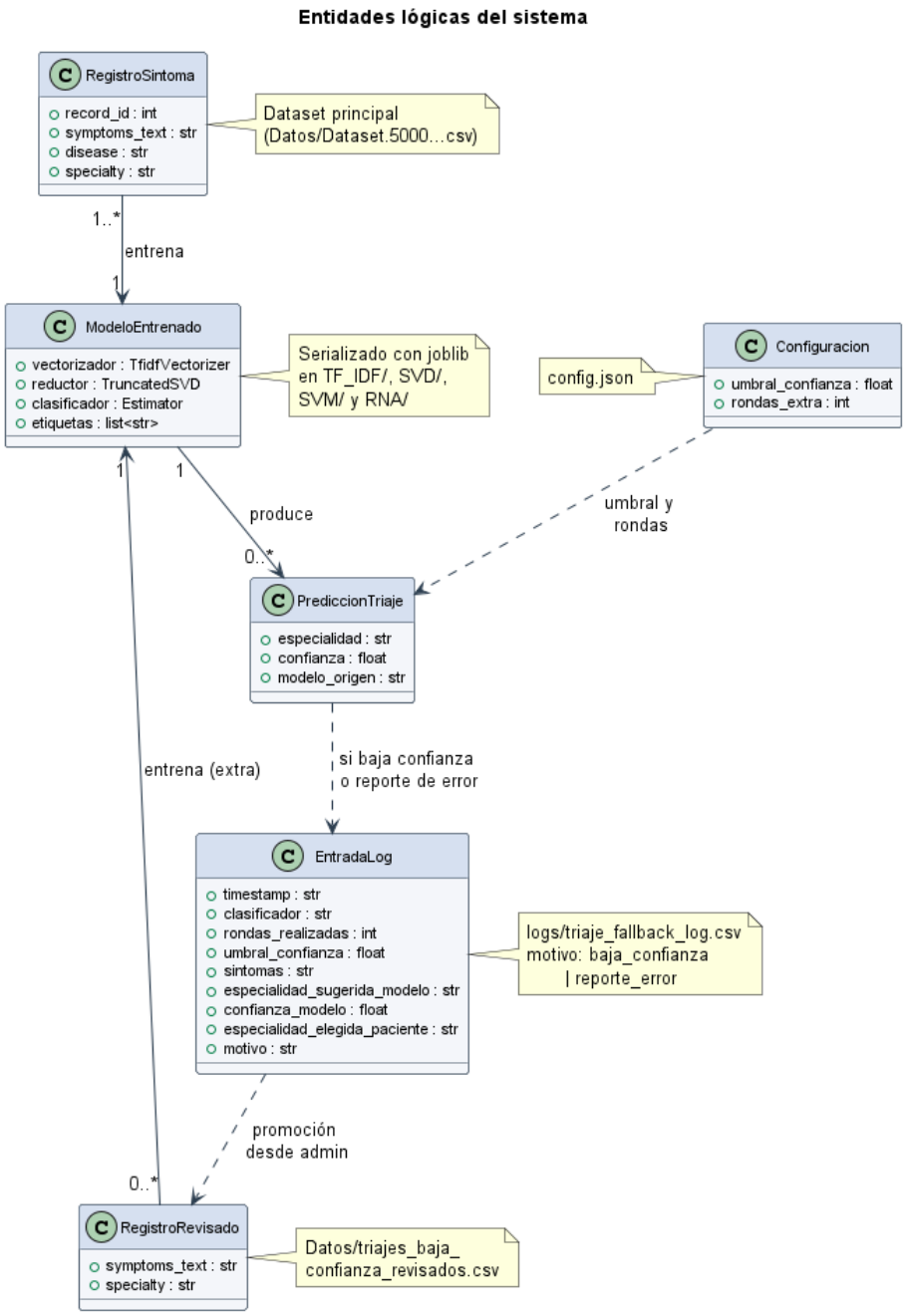

Figura C.1: Relaciones lógicas entre entidades del sistema

C.3. Diseño arquitectónico

Visión general por módulos

La aplicación se organiza alrededor de `app.py`, el punto de entrada de Streamlit y el módulo que orquesta a los demás. A su alrededor hay cuatro módulos independientes, uno por modelo, y una capa de datos compartida. La figura C.2 lo dibuja.

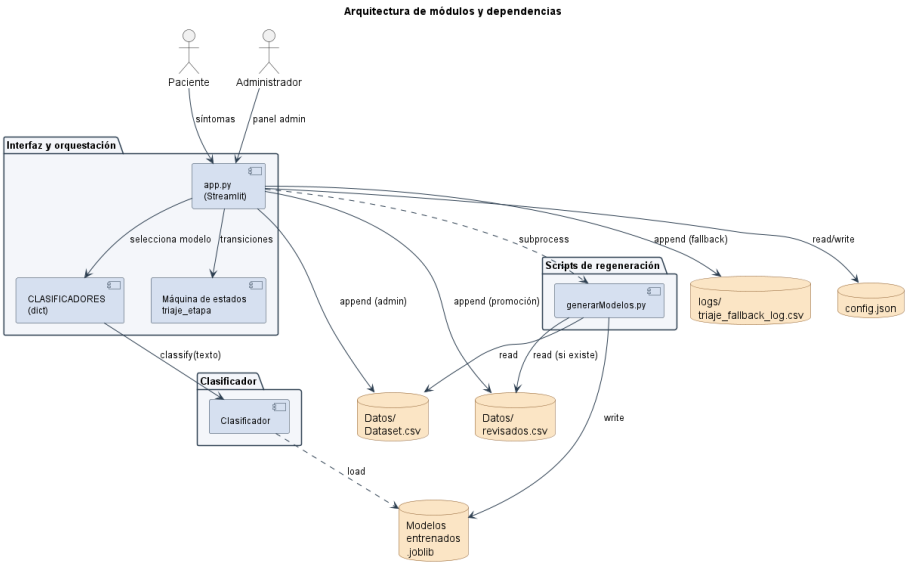

Figura C.2: Diagrama arquitectónico de módulos y dependencias

Las responsabilidades se reparten así:

- **Interfaz y orquestación (`app.py`)**: tiene las dos páginas de la aplicación Streamlit —triaje y administración—, que se conmutan desde la barra lateral con un `st.sidebar.radio`. Aquí vive también la máquina de estados del triaje, el formulario de *login* del administrador, la captura de síntomas por voz con `streamlit-mic-recorder`, la lectura y escritura de `config.json`, el log de fallback y la presentación de los resultados.

- **Módulos de clasificación** (`TF_IDF/`, `SVD/`, `SVM/`, `RNA/`) con una función que recibe un texto y devuelve la tupla `(especialidad, confianza)`.

- **Scripts de regeneración**: cada carpeta de modelo lleva además un script `generarModelo<NOMBRE>.py` que se ejecuta por separado, lee el dataset base concatenado con el CSV de revisados, entrena el modelo y lo vuelve a guardar en disco con `joblib`.
- **Datos compartidos** (`Datos/`, `logs/`, `config.json`): los datos, las correcciones revisadas, el log de fallback y la configuración del triaje. Lo leen y escriben tanto `app.py` como los scripts de regeneración.

Modo Todos: ejecutar los cuatro modelos a la vez

Por defecto, el paciente no escoge un modelo concreto: la interfaz ofrece la opción *Todos (recomendado)*, que ejecuta los cuatro clasificadores sobre los mismos síntomas y se queda con la predicción de mayor confianza. La función `ejecutar_todos_clasificadores` recorre el diccionario `CLASIFICADORES`, llama a cada uno y construye un diccionario de resultados; `mejor_resultado` se queda con el más confiado entre los que no han devuelto error.

El paciente conserva la opción de forzar un modelo concreto del `selectbox`, lo cual resulta útil durante la validación: permite ver qué pasa con cada motor por separado sobre los mismos síntomas.

Configuración persistente del triaje

Dos parámetros son configurables y se persisten en `config.json`: el umbral de confianza (45 % por defecto) y el número de rondas extra de ampliación (1 por defecto). La pestaña de administración expone un formulario para cambiarlos y al guardar invoca `guardar_config`, que sobrescribe el JSON.

Si el fichero no existe o está corrupto, `cargar_config` aplica los valores por defecto sin abortar la aplicación.

Autenticación de la pestaña de administración

La pestaña de administración está protegida por un mecanismo de autenticación sencillo, implementado por completo dentro de `app.py`. La contraseña esperada se busca, por este orden, en `st.secrets`, después en el fichero `.streamlit/secrets.toml.aos006` y, si no aparece en ninguno de los dos, en la variable de entorno `TRIAJE_ADMIN_PASSWORD`. La comparación se hace tanto sobre el nombre de usuario como sobre la contraseña, usando `hmac.compare_digest` en vez de un `==` normal, para no

ser vulnerable a ataques por tiempo. El estado de sesión se guarda en `st.session_state.admin_autenticado` y se puede cerrar desde un botón *Cerrar sesión* en la barra lateral.

Arquitectura de despliegue

El despliegue es muy simple: un único proceso Python ejecuta Streamlit, que sirve la interfaz en un puerto local (por defecto el 8501). Los modelos se cargan cuando hace falta y la aplicación trabaja con los CSV alojados en una carpeta del propio equipo. Sin servicios externos, sin conexiones de red salientes, sin nada en la nube: los datos del paciente no salen de la máquina, que es la idea standalone planteada en la memoria y la condición no funcional del anexo B.

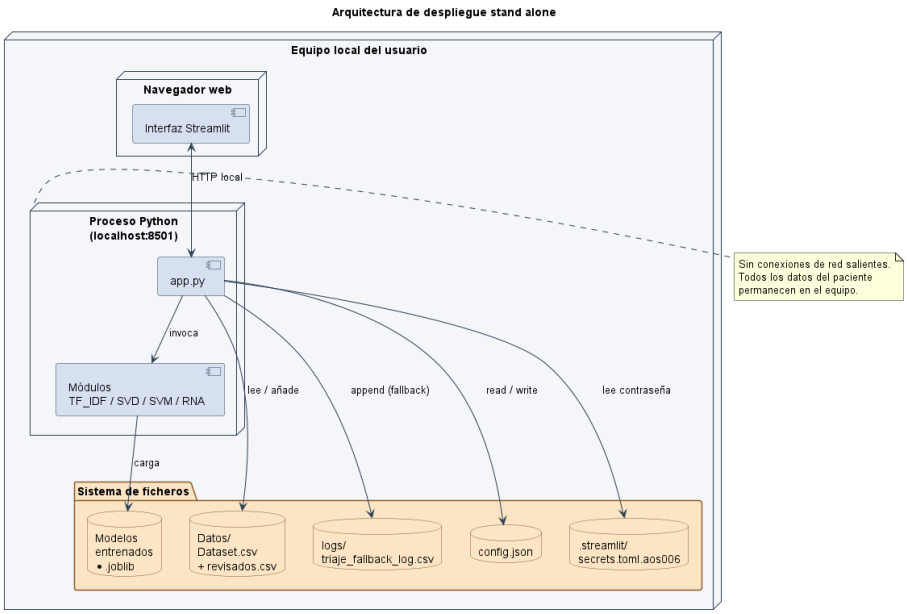

Figura C.3: Arquitectura de despliegue del prototipo

C.4. Diseño procedimental

Lo que sigue son diagramas de secuencia de los flujos más representativos: primero el triaje, que es el más complejo, y después las operaciones del panel de administración.

Máquina de estados del triaje

El flujo de triaje no es lineal: una predicción puede cumplir el umbral a la primera, requerir una o varias rondas de ampliación, terminar en fallback con selección manual del paciente, o pasar por un reporte de error tras un resultado aparentemente bueno. Para que todo esto encaje, `app.py` implementa el triaje como una pequeña máquina de estados sobre `st.session_state.triaje_etapa`, con cinco estados:

- `inicio`: el paciente introduce los síntomas y elige modelo (o *Todos*).
- `ampliando`: la confianza está por debajo del umbral y todavía quedan rondas; la interfaz pide más síntomas.
- `fallback`: se han agotado las rondas; el paciente acepta la sugerencia del modelo o elige manualmente la especialidad.
- `completado`: se muestra el resultado final y, si procede, se ofrece la opción de reportar error.
- `reportar_error`: el paciente indica que la sugerencia no era correcta y aporta la especialidad real.

Triaje con confianza suficiente (CU-01)

Es el caso ideal: el paciente introduce síntomas, el modelo predice por encima del umbral y todo termina en una sola pasada. La figura C.4 muestra la secuencia.

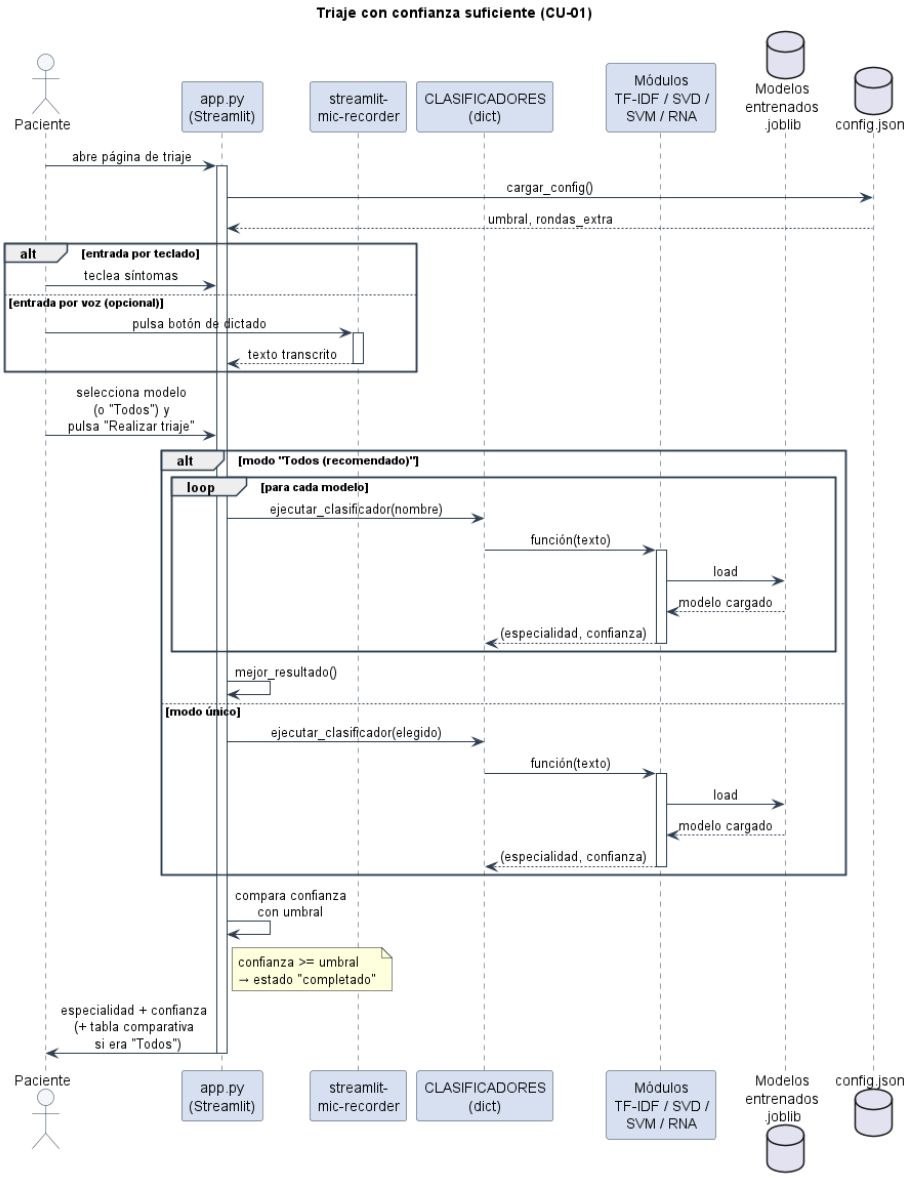

Figura C.4: Diagrama de secuencia - Triaje con confianza suficiente

Los pasos son estos:

1. El paciente entra en la página de triaje e introduce los síntomas.
2. Elige el motor en el `selectbox`. Por defecto está seleccionado *Todos (recomendado)*.

3. Pulsa el botón *Realizar triaje*. `app.py` comprueba que el campo no esté vacío y, según la opción, invoca un único modelo o los cuatro a la vez.

4. Cada módulo carga su modelo serializado con `joblib`, preprocesa el texto y devuelve `(especialidad, confianza)`.

5. `app.py` compara la confianza con el umbral configurado. Si lo supera, transita al estado `completado`.

6. En `completado`, se muestra la especialidad sugerida con su porcentaje de confianza. Si el modo era *Todos*, se muestra además la tabla comparativa de los cuatro modelos. La pantalla ofrece dos botones: *Realizar otro triaje* y *Reportar error*.

Triaje con ampliación de síntomas (CU-02)

Cuando la confianza de la primera predicción está por debajo del umbral y quedan rondas extra disponibles, la máquina de estados transita a `ampliando`. La figura C.5 muestra el flujo.

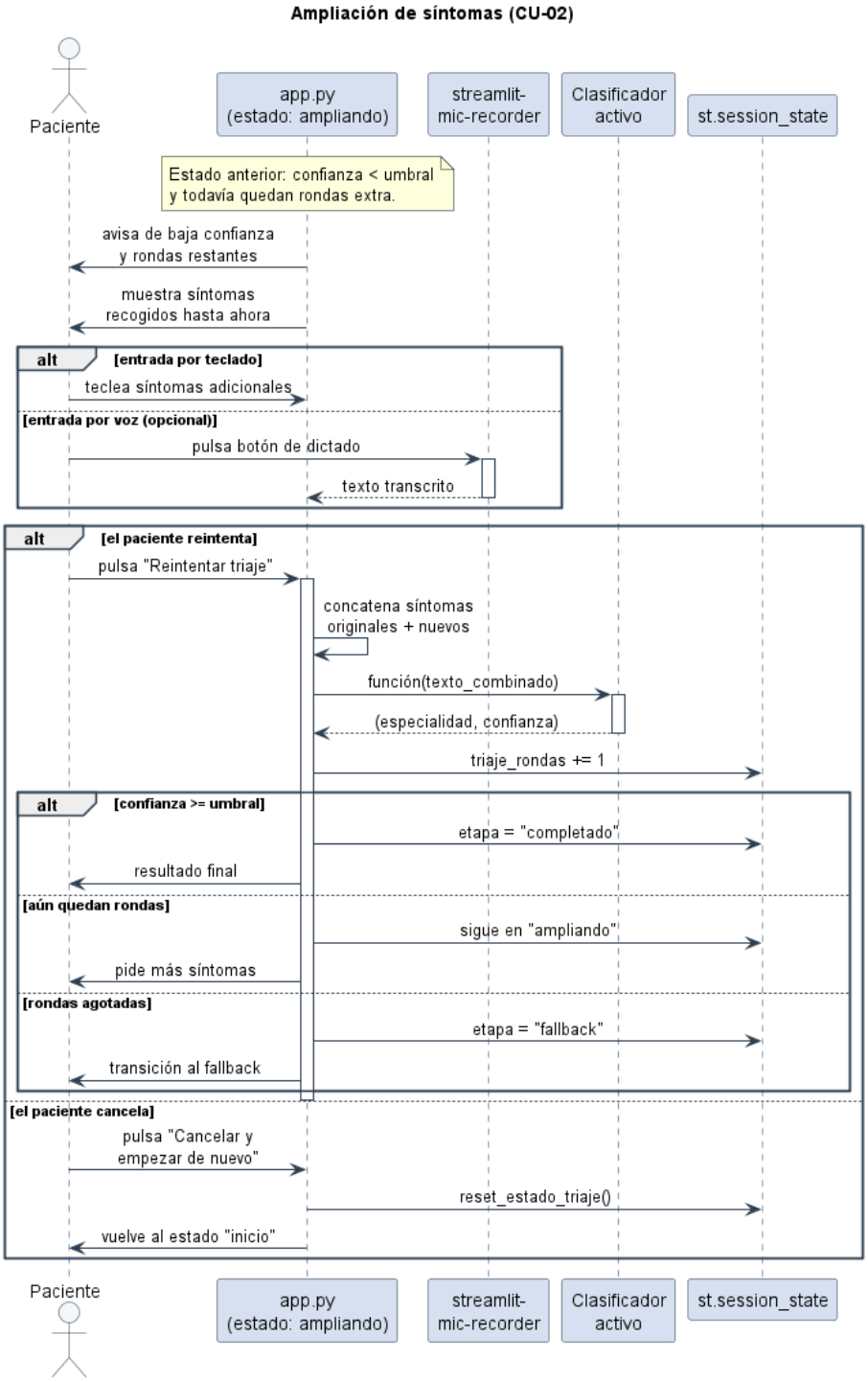

Figura C.5: Diagrama de secuencia - Ampliación de síntomas

En este estado, `app.py` avisa al paciente de que la confianza fue insuficiente, le indica cuántas rondas le quedan y pide síntomas adicionales. Al pulsar *Reintentar triaje*, la aplicación concatena los síntomas originales con los nuevos, vuelve a invocar al mismo clasificador y compara de nuevo:

- Si la confianza ya supera el umbral, transita a `completado`.
- Si todavía no lo supera pero quedan rondas, sigue en `ampliando`.
- Si se agotan las rondas, pasa al estado `fallback`.

En cualquier momento, el paciente puede pulsar *Cancelar y empezar de nuevo*, que resetea el estado y vuelve a `inicio`.

Fallback con selección manual del paciente (CU-03)

Cuando se agotan las rondas sin alcanzar el umbral, la aplicación entra en el estado `fallback`. En este punto deja de intentar una predicción automática y pasa la decisión al paciente. La figura C.6 muestra el flujo.

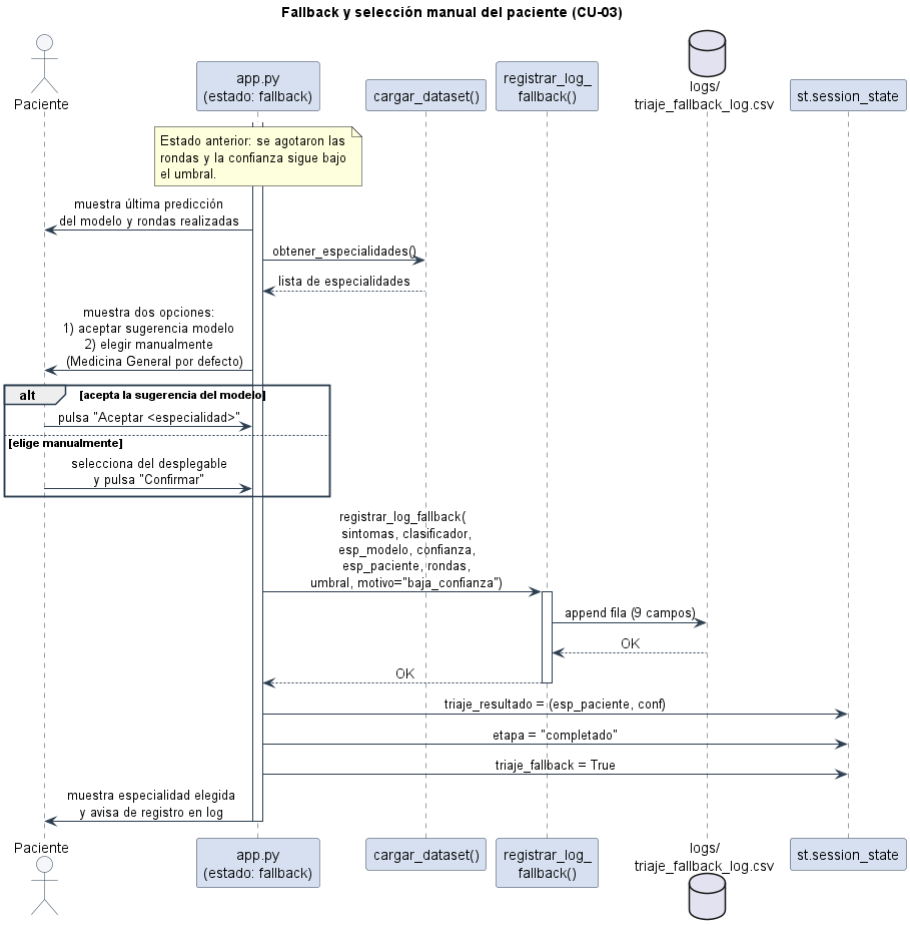

Figura C.6: Diagrama de secuencia - Fallback y selección manual

La interfaz ofrece dos opciones. La primera es aceptar la sugerencia del modelo aunque no haya llegado al umbral. La segunda es elegir manualmente la especialidad desde un desplegable con todas las que ya existen en el dataset; por defecto aparece *Medicina General*.

Al confirmar cualquiera de las dos opciones, `app.py` invoca `registrar_log_fallback`, que añade una fila al log con todos los datos del intento: marca temporal, clasificador, rondas realizadas, umbral aplicado, síntomas, especialidad del modelo, confianza, especialidad elegida por el paciente y motivo `baja_confianza`. Después transita al estado `completado`, que muestra la especialidad elegida y avisa de que el caso ha quedado registrado para revisión.

Reporte de error tras un acierto aparente (CU-03b)

Hay un escenario que el umbral por sí solo no cubre: el modelo predice con alta confianza una especialidad equivocada. Para esos casos, el estado `completado` ofrece un botón *Reportar error* que transita al estado `reportar_error`.

La interfaz funciona como la del fallback: desplegable con todas las especialidades y botones de enviar y cancelar. Lo que cambia es que la fila que se añade al log lleva motivo `reporte_error`, para diferenciar en la revisión posterior los casos en los que el modelo predijo bajo de confianza de los casos en los que predijo seguro pero falló.

Añadir nuevos síntomas desde administración (CU-04)

La pestaña de administración deja al usuario autenticado incorporar nuevos pares de síntomas y especialidad los datos de base.Es un formulario que tiene un campo de texto libre para los síntomas, dos vías para fijar la especialidad —elegirla del `selectbox` de las ya existentes o teclear una nueva— y un campo opcional para la enfermedad asociada.

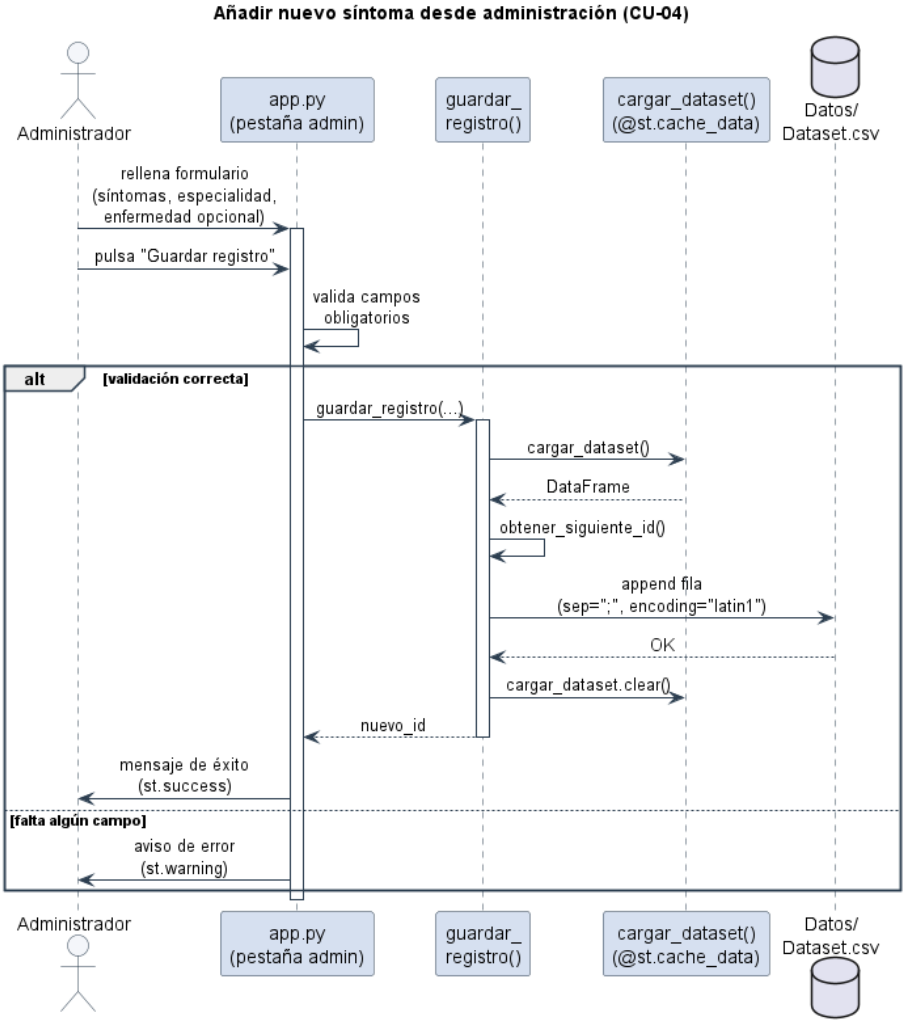

Figura C.7: Diagrama de secuencia - Añadir síntoma desde administración

Al pulsar *Guardar registro*, `app.py` valida los campos obligatorios, calcula el siguiente identificador a partir del máximo actual de `record_id` y anexa la fila directamente al CSV base usando el módulo `csv` estándar, respetando el separador y la codificación originales.

Revisar el log y promover casos al CSV de revisados (CU-05)

Una sección del panel de administración muestra el log de fallback como una lista de tarjetas. Cada tarjeta corresponde a un triaje fallido o reportado como erróneo, y permite al administrador revisar los síntomas, ajustar la especialidad propuesta si lo cree conveniente y, con un solo botón, promover ese caso al CSV de revisados.

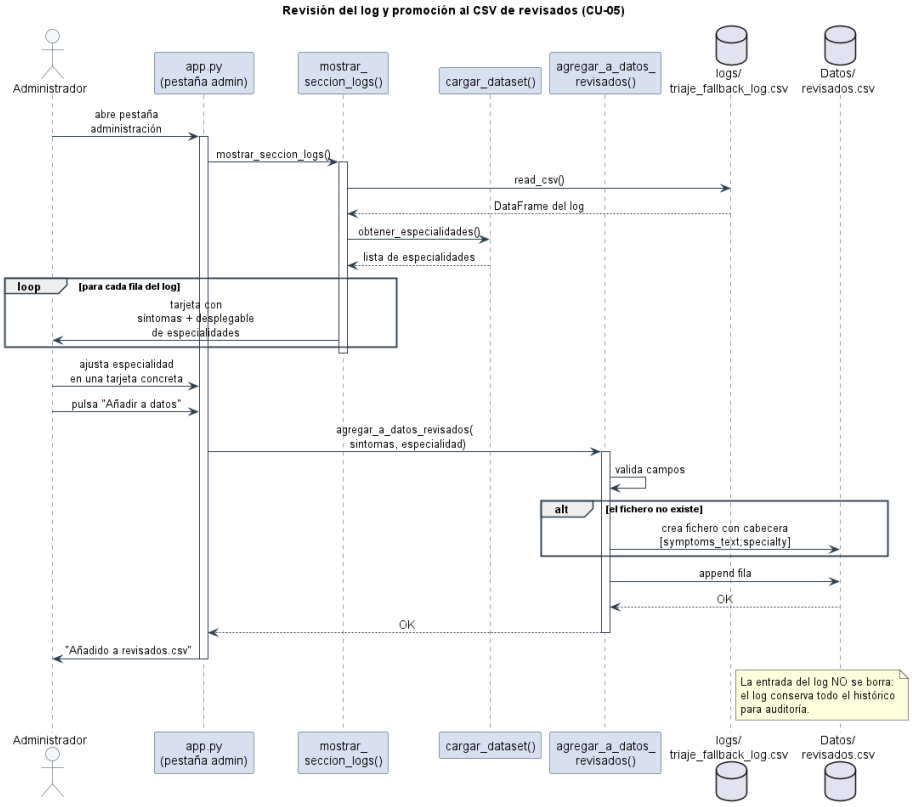

Figura C.8: Diagrama de secuencia - Revisión del log y promoción al CSV de revisados

La actualización la hace `agregar_a_datos_revisados`, que añade una fila con solo *symptoms_text* y *specialty* al fichero `Datos/triajes_baja_confianza_revisados.csv`. La entrada del log no se borra: el log conserva el historial completo para auditoría.

Regeneración de modelos (CU-06)

Es la operación más costosa de la aplicación: vuelve a entrenar cada modelo sobre el corpus actualizado. La pestaña de administración ofrece dos vías. Un botón principal *Regenerar todos los modelos* dispara los cuatro scripts en secuencia, mostrando una barra de progreso o cuatro botones individuales.

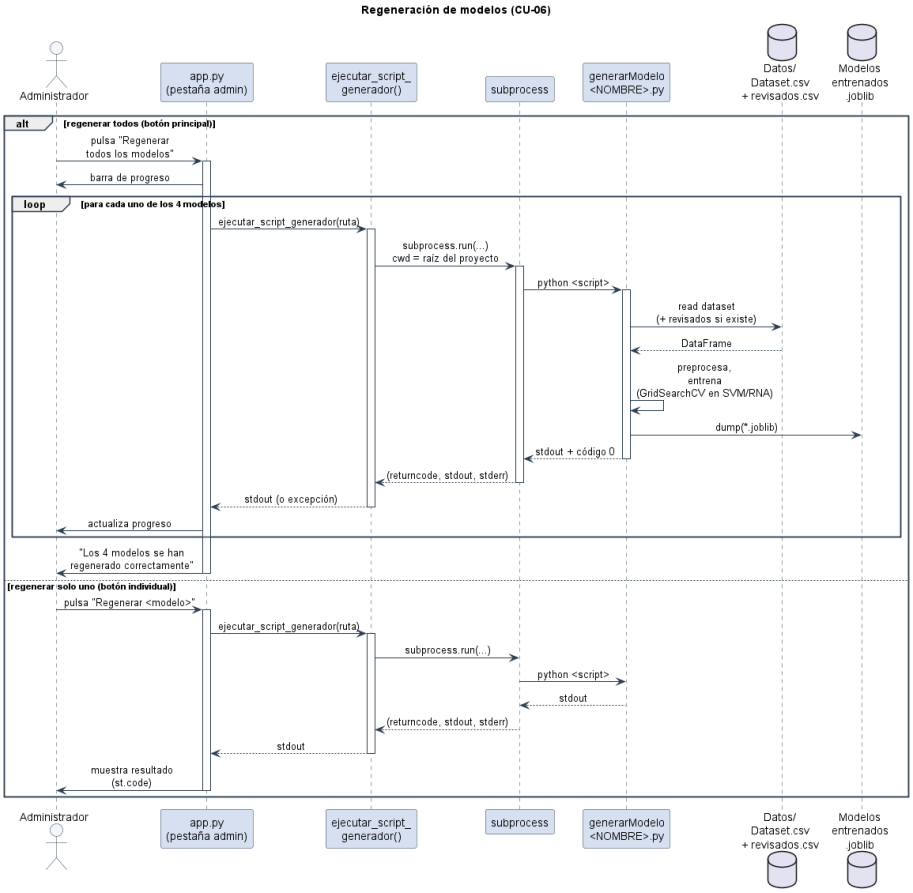

Figura C.9: Diagrama de secuencia - Regeneración de los modelos

Todo el lanzamiento vive en la función `ejecutar_script_generador`:

1. Comprueba que el script existe en la ruta esperada (por ejemplo, `TF_IDF/generarModeloTFIDF.py`).

2. Lo lanza con `subprocess.run` usando `sys.executable`, fijando `cwd` en la raíz del proyecto para que las rutas relativas dentro del script funcionen.

3. Captura la salida estándar y la de error; si el código de retorno no es cero, lanza una excepción con el contenido capturado.

4. Si todo va bien, devuelve la salida estándar para que `app.py` la pinte en un bloque `st.code`.

Acceso autenticado al panel de administración (CU-07)

La autenticación se ejecuta antes de pintar nada del panel. Cuando el usuario elige `Administración` en la barra lateral, `app.py` mira `st.session_state.admin_autenticado`; si la sesión todavía no está autenticada, aparece el formulario con usuario y contraseña.

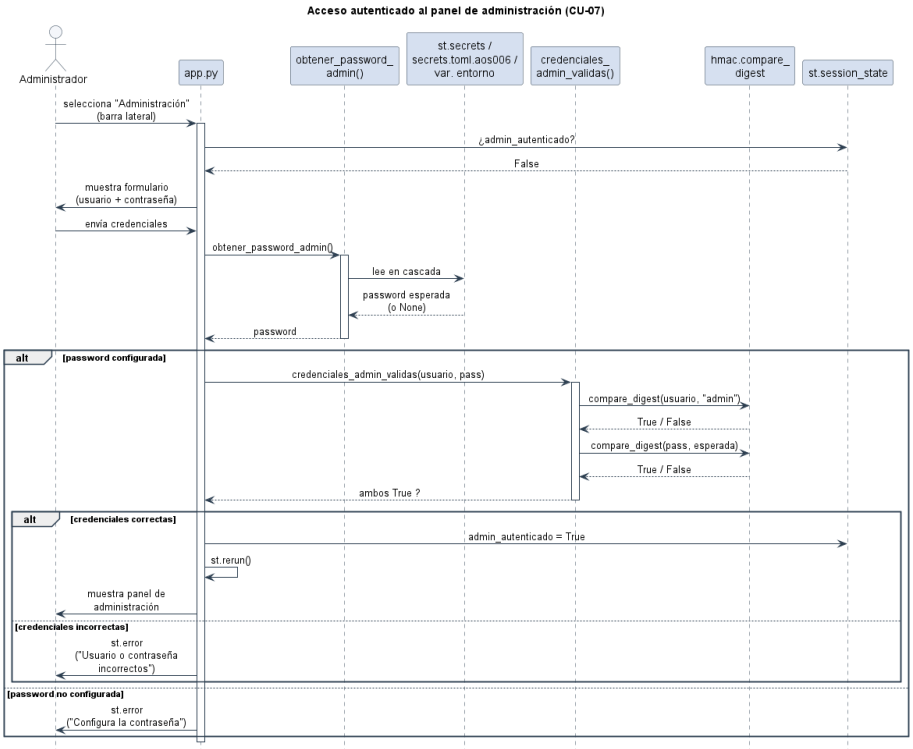

Figura C.10: Diagrama de secuencia - Acceso al panel de administración

Al enviar el formulario, la función `credenciales_admin_validas` compara con `hmac.compare_digest` tanto el nombre de usuario como la con-

traseña. Si ambos coinciden, marca la sesión como autenticada y llama a `st.rerun` para repintar la página, ya con el contenido administrativo. Si no, muestra error y el formulario sigue visible. Y si la contraseña no está configurada en ningún sitio, la aplicación avisa expresamente, para que un despliegue mal configurado no deje el panel abierto.

Apéndice D

Documentación técnica de programación

D.1. Introducción

Este apéndice está dirigido a quien quiera entender cómo está montado el proyecto por dentro, ponerlo a funcionar en su propio equipo y, si llega el caso, modificarlo. No es un manual de uso; eso queda para el apéndice del usuario.

La aplicación es un prototipo de triaje médico escrito en Python que corre en local sobre Streamlit [7]. Recibe síntomas en lenguaje natural (escritos a mano o dictados por voz) y devuelve una especialidad médica. Por debajo trabajan cuatro modelos en paralelo: TF-IDF con Regresión Logística, TF-IDF combinado con SVD, SVM y una red neuronal de tipo MLP. Por defecto, los cuatro se ejecutan sobre los mismos síntomas y la predicción que se muestra al paciente es la del que devuelve mayor confianza, junto con una tabla comparativa de los cuatro. Cada modelo se entrena por separado y se guarda en disco como un fichero `.joblib`, así la aplicación lo carga en memoria cuando lo necesita sin tener que reentrenar nada.

En las páginas siguientes se explica cómo está repartido el código, qué herramientas hacen falta para preparar el entorno, cómo se descarga el proyecto desde GitHub, cómo se entrenan los modelos y se arranca la aplicación, y por último qué pruebas se han hecho.

D.2. Estructura de directorios

El proyecto está organizado con una carpeta por modelo y los datos y los logs separados del código. Así se puede tocar un modelo sin afectar a los demás y los ficheros que la propia aplicación genera durante su uso quedan recogidos en su sitio.

- `/`: en la raíz están `app.py`, que es el punto de entrada de la aplicación Streamlit; `config.json`, con el umbral de confianza y las rondas de ampliación; `requirements.txt`, con las dependencias; el `README.md` y el `.gitignore`.

- `/Datos`: contiene el conjunto de entrenamiento principal (`Dataset.5000.Registros.Marz.ID.Sintomas.Enfermedad.Especialidad.csv`) y el CSV ampliado (`triajes_baja_confianza_revisados.csv`), que va recogiendo los síntomas revisados desde el panel de administración (los que llegan por baja confianza y también los reportes de error del paciente). Cuando se vuelven a entrenar los modelos, los scripts concatenan ambos ficheros, así las correcciones entran en el siguiente entrenamiento sin tocar el dataset original.

- `/TF_IDF`: contiene `tf_idf.py` con la función de inferencia y `generarModeloTFIDF.py`, que entrena el modelo base de TF-IDF con Regresión Logística. El modelo entrenado se guarda como `tfidf_model.joblib` en la misma carpeta.

- `/SVD`: contiene `SVD.py` y `generarModeloSVD.py`. El entrenamiento aplica TF-IDF, luego TruncatedSVD con 100 componentes y, al final, un clasificador de Regresión Logística. El resultado queda en `svd_model.joblib`.

- `/SVM`: contiene `SVM.py` y `generarModeloSVM.py`. El entrenamiento monta un `Pipeline` de scikit-learn con `GridSearchCV` para ajustar el parámetro `C` y el tamaño del vocabulario. El modelo se guarda en `svm_model.joblib`.

- `/RNA`: contiene `RNA.py` y `generarModeloRNA.py`. El entrenamiento utiliza un MLP con *early stopping* y también pasa por `GridSearchCV` para probar varias arquitecturas y funciones de activación. El modelo final queda en `rna_model.joblib`.

- `/logs`: aquí se guarda `triaje_fallback_log.csv`, donde acaban tanto las consultas que el sistema no ha clasificado con confianza suficiente como los reportes de error que envía el paciente cuando considera que la especialidad sugerida no era correcta. Cada fila guarda el modelo usado, los síntomas, la especialidad sugerida por el modelo, la que finalmente eligió el paciente y un campo `motivo` para distinguir un caso del otro.
- `/.streamlit`: contiene la configuración de Streamlit y el fichero de *secrets* con la contraseña del administrador. Este fichero queda fuera del control de versiones.
- `/tex` y `/img`: contienen los ficheros LaTeX y las imágenes de la memoria y los anexos.

D.3. Manual del programador

Herramientas necesarias

Para montar un entorno de desarrollo equivalente al que he utilizado durante el TFG hace falta lo siguiente:

- **Python 3.11 o superior**. Es el lenguaje en el que está escrito todo el código.
- **Git**, para clonar el repositorio y llevar el control de versiones.
- **Visual Studio Code** (recomendado), aunque sirve cualquier otro editor con soporte para Python.
- **Una cuenta de GitHub**, opcional, si se quiere contribuir al repositorio o usar *GitHub Copilot* como ayuda durante la programación.

Python

Toda la aplicación se ha desarrollado en Python 3.11. Se puede descargar desde python.org/downloads. En Windows conviene marcar durante la instalación la opción *Add Python to PATH*, así se pueden ejecutar `python` y `pip` desde cualquier terminal.

Una vez instalado, conviene comprobar la versión:

```
python --version
pip --version
```

Visual Studio Code

Como editor he usado Visual Studio Code. Una vez instalado, desde el panel de extensiones (`Ctrl+Shift+X`) conviene añadir:

- **Python** (Microsoft): aporta el intérprete, el depurador y la integración con entornos virtuales.
- **Pylance**: análisis estático y autocompletado.
- **GitLens**: para consultar el historial de cambios sin salir del editor.
- **GitHub Copilot** (opcional): asistencia durante la escritura del código. Es gratuito para estudiantes a través de *GitHub Education*.

Git

Git se descarga desde git-scm.com. En Windows lo más cómodo es dejar las opciones por defecto del asistente, ya que deja preparados tanto *Git Bash* como la terminal de Windows.

Obtención del código fuente

El código está en un repositorio público de GitHub [4]. Para descargarlo basta con:

```
git clone https://github.com/<usuario>/<repositorio>.git
cd <repositorio>
```

Tras el clonado aparece la estructura de directorios descrita más arriba.

Entorno virtual y dependencias

Para no interferir con otros proyectos de Python que pueda haber en el equipo, conviene crear un entorno virtual antes de instalar nada. Desde la raíz del proyecto:

```
python -m venv .venv
```

Luego se activa. En Windows, desde *PowerShell*:

```
.venv\Scripts\Activate.ps1
```

Y en Linux o macOS:

```
source .venv/bin/activate
```

Una vez activo el entorno (el prompt mostrará `(.venv)` al inicio), se instalan las librerías. El repositorio incluye un `requirements.txt` con las dependencias mínimas (`streamlit`, `streamlit-mic-recorder`, `scikit-learn`, `pandas`, `nltk` y `joblib`):

```
pip install -r requirements.txt
```

La primera vez que se ejecute el código de los modelos, NLTK descargará el corpus de palabras vacías en español:

```
import nltk
nltk.download('stopwords')
```

La descarga se hace una sola vez por equipo y la lanzan los propios módulos.

Configuración del acceso de administrador

La pestaña de administración está protegida con contraseña. La aplicación la busca por este orden:

1. En el valor `admin_password` de los *secrets* de Streamlit.
2. En el fichero `.streamlit/secrets.toml.aos006`.
3. En la variable de entorno `TRIAJE_ADMIN_PASSWORD`.

Lo más sencillo para una instalación local es crear el fichero `.streamlit/secrets.toml` con:

```
admin_password = "tu_contraseña"
```

Este fichero está en el `.gitignore`, así que la contraseña no sube nunca a GitHub. El usuario es siempre `admin`.

D.4. Compilación, instalación y ejecución del proyecto

Python no tiene fase de compilación, así que poner la aplicación en marcha consiste en preparar los datos, entrenar los modelos y arrancar Streamlit.

Entrenamiento de los modelos

Antes de usar la aplicación tienen que existir los cuatro ficheros `.joblib`. Cada modelo tiene su propio script de entrenamiento que se lanza por separado. Desde la raíz del proyecto y con el entorno virtual activo:

```
python TF_IDF/generarModeloTFIDF.py
python SVD/generarModeloSVD.py
python SVM/generarModeloSVM.py
python RNA/generarModeloRNA.py
```

Los dos primeros (TF-IDF y SVD) terminan en pocos segundos, porque utilizan Regresión Logística sin búsqueda de hiperparámetros. Los de SVM y red neuronal son más lentos, porque pasan por GridSearchCV sobre validación cruzada estratificada de cinco particiones. En un equipo modesto, el entrenamiento de la red neuronal puede tardar varios minutos.

Quien prefiera no entrar a la línea de comandos puede usar la pestaña de administración: hay un botón individual por modelo y un botón *Regenerar todos los modelos* que lanza los cuatro scripts en secuencia con una barra de progreso. Por dentro, los botones lanzan el mismo script con `subprocess.run` y muestran su salida en la interfaz.

Los cuatro scripts hacen lo mismo:

1. Cargan el CSV principal con `pandas.read_csv` y, si existe, lo concatenan con el ampliado `triajes_baja_confianza_revisados.csv`. Así los registros revisados desde la administración entran en el entrenamiento sin tocar el dataset original.

2. Aplican el preprocesado de texto: paso a minúsculas, eliminación de caracteres no alfabéticos, retirada de palabras vacías y Stemming con `SnowballStemmer`.

3. Entrenan el modelo correspondiente.
4. Guardan el resultado con `joblib.dump`, en forma de tupla (vectorizador, clasificador) o, en el caso de SVD, (vectorizador, SVD, clasificador).

Ejecución de la aplicación

Una vez generados los modelos, la aplicación se lanza con:

```
streamlit run app.py
```

Streamlit arranca un servidor local y abre el navegador por defecto en `http://localhost:8501`. Hay dos pestañas, accesibles desde la barra lateral:

- **Triaje**: la vista del paciente. Permite escribir o dictar síntomas. Por defecto, la opción del desplegable es *Todos (recomendado)*: la aplicación lanza los cuatro modelos sobre la misma descripción y muestra una tabla comparativa con sus predicciones, marcando al de mayor confianza. Como alternativa, se puede elegir un modelo concreto y prescindir de la comparativa.
- **Administración**: protegida por contraseña. Sirve para añadir registros al dataset ampliado, revisar el log de consultas problemáticas (baja confianza y reportes de error), ajustar el umbral y las rondas de ampliación, y regenerar los modelos uno a uno o todos a la vez.

Configuración del umbral y las rondas

El comportamiento del triaje se controla desde `config.json`, en la raíz del proyecto. El contenido por defecto es:

```
{
 "umbral_confianza": 45.0,
 "rondas_extra": 1
}
```

`umbral_confianza` es el porcentaje mínimo de confianza para dar por buena la predicción. `rondas_extra` dice cuántas veces se le va a pedir al

paciente que añada más síntomas antes de pasar al fallback (selección manual o derivación a Medicina General).

Los dos valores se cambian desde la pestaña de administración con un formulario, y el cambio se aplica sin reiniciar la aplicación.

Flujo de un triaje

El desplegable de selección de motor de IA arranca en *Todos (recomendado)*, así que el comportamiento por defecto cuando el paciente pulsa el botón de triaje es el siguiente:

1. La función `ejecutar_todos_clasificadores` llama una vez a cada uno de los cuatro modelos (`tf_idf`, `svd`, `svm` y `rna`) con los mismos síntomas. Cada función carga su `.joblib` con `joblib.load` y devuelve una tupla `(especialidad, confianza)`.

2. `mejor_resultado` se queda con el modelo que ha devuelto mayor confianza. Ese es el que se muestra como predicción principal.

3. En la pantalla de resultado aparece una tabla generada con `pd.DataFrame` que recoge la predicción y la confianza de los cuatro modelos. El ganador queda marcado con un asterisco y la etiqueta *Elegido*. Si alguno de los modelos falla, su fila aparece con estado *Error*.

4. Si la confianza del ganador supera el umbral configurado, el resultado se da por bueno.

5. Si queda por debajo y todavía hay rondas de ampliación disponibles, la aplicación pide al paciente que añada más síntomas. En la ampliación ya no se vuelven a ejecutar los cuatro modelos: solo se reintenta con el que había ganado en la primera ronda, hasta que su confianza supere el umbral o se agoten las rondas.

6. Si tras agotar las rondas la confianza sigue siendo baja, se entra en *fallback*: el paciente puede aceptar la sugerencia del modelo, elegir manualmente otra especialidad de la lista o quedarse con Medicina General. En cualquiera de los tres casos, la consulta se registra en `logs/triaje_fallback_log.csv` con motivo `baja_confianza`.

7. Aunque la confianza del ganador haya superado el umbral, en la pantalla de resultado siempre aparece un botón *Reportar error*. Si el paciente lo pulsa, indica la especialidad que considera correcta y la

consulta se añade al mismo log, esta vez con motivo `reporte_error`. Así se recogen correcciones incluso cuando el sistema estaba seguro de su predicción.

El desplegable también ofrece los cuatro modelos por separado como opción minoritaria, pensada para hacer pruebas o comparar comportamientos puntuales. En ese caso la lógica es la misma, pero solo se invoca el modelo elegido y la tabla comparativa no aparece.

Las entradas del log (por baja confianza o por reporte de error) se revisan después desde la administración. Cuando el administrador asigna una especialidad y pulsa *Añadir a datos*, el par síntomas-especialidad se vuelca en `triajes_baja_confianza_revisados.csv`, que es el CSV que se concatena con el principal al reentrenar.

Entrada por voz

Como alternativa al teclado, el paciente puede dictar los síntomas. La captura del audio la hace el componente `streamlit-mic-recorder` [2] directamente en el navegador, y la transcripción que devuelve se inserta en el cuadro de síntomas. De ahí en adelante, el flujo es idéntico al de un texto escrito a mano.

Para que funcione, el navegador tiene que dar permiso al micrófono. La primera vez aparece una ventana de confirmación; Chrome y Firefox lo recuerdan en sesiones posteriores siempre que se entre desde `localhost`.

D.5. Pruebas del sistema

El proyecto es un prototipo académico que corre en local y cuya interfaz ha cambiado bastante durante el desarrollo, así que las pruebas se han hecho de forma mixta: pruebas manuales sobre la interfaz Streamlit y validación de los modelos durante el entrenamiento. No se ha montado una batería de tests automatizados con `pytest` dentro del alcance del TFG, aunque es uno de los puntos pendientes.

Pruebas de los modelos durante el entrenamiento

La evaluación principal de los cuatro modelos se hace dentro del propio entrenamiento. Los scripts `generarModeloSVM.py` y `generarModeloRNA.py` usan GridSearchCV con validación cruzada estratificada de cinco particiones,

que da una estimación razonable del rendimiento de cada combinación de hiperparámetros sin tener que reservar un conjunto de test fijo.

Las métricas son las habituales en clasificación supervisada multiclase: *accuracy*, *precision*, *recall* y *F1-macro*. La versión macro se usa en lugar de la micro porque el dataset está desequilibrado: hay muchos más ejemplos de atención general y medicina interna que de especialidades como dermatología o neurología, y el macro da a todas las clases el mismo peso.

Los modelos TF-IDF base y SVD funcionan como referencia. Por eso se entrenan con una configuración fija, sin búsqueda exhaustiva, y se evalúan con la división entrenamiento/test que hace `train_test_split` reservando un 20 % para test.

Pruebas manuales sobre la interfaz

A lo largo del desarrollo he ido probando manualmente los escenarios que más se repiten:

- **Triaje en modo conjunto (escenario por defecto)**: con el desplegable en *Todos (recomendado)*, lanzar descripciones de distinto tipo y comprobar que la tabla muestra los cuatro modelos, que el asterisco recae sobre el de mayor confianza y que la predicción que se muestra arriba coincide con la fila marcada como elegida.
- **Confianza alta**: descripciones claras como *"dolor torácico opresivo que irradia al brazo izquierdo"*, para ver que al menos uno de los cuatro modelos supera el umbral y el flujo termina sin pasar por ampliación.
- **Confianza baja**: descripciones vagas como *"me encuentro mal"* o *"dolor"*, para comprobar que ningún modelo supera el umbral, que entra la ronda de ampliación y que en esa ronda solo se reintenta con el ganador de la primera vuelta.
- **Modelo individual**: forzar cada uno de los cuatro modelos por separado desde el desplegable, sobre la misma descripción, para comparar predicciones puntuales. En este modo la tabla comparativa no aparece, lo cual también se ha verificado.
- **Reporte de error**: en un triaje con confianza alta, pulsar *Reportar error*, indicar otra especialidad y comprobar que la entrada queda en el log con motivo `reporte_error`, mientras que las del fallback aparecen con motivo `baja_confianza`.

- **Entrada por voz**: dictar síntomas en español, ver que la transcripción se inserta en el cuadro de texto y que el resto del flujo funciona igual que escribiendo a mano.
- **Acceso de administrador**: intentos con credenciales correctas e incorrectas, comprobando que la pestaña no se desbloquea hasta validar la contraseña.
- **Alta de un registro nuevo**: añadir pares síntoma-especialidad desde la administración, comprobando que se asigna el siguiente `record_id` disponible y que el registro aparece en el CSV ampliado.
- **Revisión del log**: leer el log, asignar manualmente una especialidad a una entrada (tanto de baja confianza como de reporte de error) y volcarla al CSV ampliado. He comprobado que las entradas revisadas entran luego en el siguiente reentrenamiento.
- **Regeneración de modelos**: pulsar los botones individuales y el botón *Regenerar todos los modelos*, comprobando que cada script termina sin error, que la barra de progreso avanza y que la aplicación carga los nuevos `.joblib` sin reiniciarse.

Validación con casos reales

Aparte de los escenarios anteriores, he ido recogiendo descripciones reales de familiares, amigos y consultas habituales. Son casos que no están en el conjunto de entrenamiento, así que sirven para ver cómo se porta el sistema cuando el lenguaje es coloquial, hay faltas de ortografía o las expresiones son poco precisas.

Algunas cosas que he ido viendo:

- Las descripciones cortas o demasiado genéricas dan confianza baja y la aplicación entra en la ronda de ampliación, que es la reacción que se busca.
- Los modelos con bigramas (SVM y MLP) se manejan algo mejor con expresiones del tipo *"dolor de cabeza intenso"* o *"falta de aire al subir escaleras"*.
- Las negaciones (*"no tengo fiebre"*) no se interpretan bien. La representación en bolsa de palabras no captura ese matiz, y es algo que asumo como una limitación del enfoque.

Pruebas de robustez del log y los CSV

También he provocado a propósito algunas situaciones límite para ver cómo reacciona la aplicación:

- Arrancar la aplicación con el log vacío o sin que exista. La aplicación lo crea con su cabecera la primera vez que registra una entrada.
- Logs antiguos sin la columna `motivo`. La función `_asegurar_header_log()` los detecta y los reescribe en disco con la cabecera nueva, rellenando las filas existentes con `baja_confianza`. La migración se ejecuta una sola vez y deja el fichero en un estado consistente.
- Síntomas con tildes, ñ o signos de puntuación. El dataset se escribe en Latin-1 y el log de fallback en UTF-8. En ambos casos los caracteres se conservan bien.
- Borrar o estropear `config.json`. La aplicación vuelve a los valores por defecto (`umbral_confianza = 60.0`, `rondas_extra = 1`) sin lanzar excepciones.

Mejoras pendientes en las pruebas

Hay tres cosas que se quedan sin hacer y que tendría sentido incorporar más adelante:

- Una batería de tests automatizados con `pytest` sobre las funciones de preprocesado y de carga de modelos. Son las partes más fáciles de aislar.
- Un script de evaluación con un conjunto de test fijo que genere una matriz de confusión por modelo, para poder comparar versiones del dataset de forma reproducible.
- Integración con SonarCloud u otro servicio de análisis estático, para revisar la calidad del código en cada *push* al repositorio.

No las he incluido por tiempo, pero el código está pensado para que añadirlas más adelante no obligue a una refactorización importante.

Apéndice E

Documentación de usuario

E.1. Introducción

Este apéndice no es para quien va a tocar el código, sino para quien va a usar la aplicación. Cubre dos cosas: lo que hace falta tener instalado para arrancarla y cómo se trabaja con ella una vez en marcha.

La aplicación es un prototipo de triaje médico. Recibe síntomas escritos o dictados por voz y devuelve la especialidad que mejor encaja con la descripción. Todo el proceso ocurre en el equipo del usuario; los síntomas no salen del navegador. Tiene dos vistas: la del paciente, donde se introducen los síntomas y se obtiene la especialidad, y la del administrador, donde se ajustan los parámetros del clasificador, se revisan los casos dudosos y se reentrenan los modelos.

El apéndice sigue ese mismo orden: requisitos, instalación y, por último, el manual organizado por pantallas. La idea es que pueda leerse con la aplicación abierta delante.

E.2. Requisitos de usuarios

La aplicación se ejecuta en un equipo personal sin necesidad de servidores externos ni conexión permanente a internet, aunque hay que tener instaladas algunas herramientas. El paciente solo necesita un navegador y, si quiere dictar los síntomas, un micrófono. El administrador no necesita nada adicional aparte de la contraseña configurada en el equipo.

Requisitos del sistema operativo

El prototipo se ha probado sobre Windows 10 y Windows 11, pero funciona igual en Linux y en macOS recientes, porque toda la pila (Python, Streamlit, scikit-learn) es multiplataforma. No hay una versión mínima del sistema operativo más allá de poder instalar Python 3.10 o superior.

Requisitos de software

Antes de instalar el proyecto hay que tener:

- **Python 3.10 o superior**. El desarrollo se ha hecho con la 3.11. Se descarga desde `https://www.python.org/downloads/`. En Windows conviene marcar *Add Python to PATH* durante la instalación, porque luego ahorra problemas con los comandos.
- **Pip**, que viene incluido con la instalación estándar de Python y se usa para instalar las dependencias.
- **Git**, opcional. Es la forma cómoda de bajarse el código, pero también se puede descargar el repositorio como ZIP desde la web de GitHub.
- **Un navegador moderno**: Chrome, Firefox o Edge en una versión razonablemente actual. La aplicación se abre como una pestaña normal.

Requisitos de hardware

El prototipo sigue la filosofía de IA Frugal, así que no pide gran cosa. Como referencia:

- Procesador x86_64 de 64 bits, tipo Intel Core i3 o equivalente.
- 4 GB de RAM como mínimo. Con 8 GB el reentrenamiento de los cuatro modelos en una misma sesión va más holgado.
- Unos 500 MB de disco entre código, dependencias y los ficheros con los modelos.
- Micrófono, solo si se quiere usar la entrada por voz. Sin él, la aplicación funciona escribiendo.

No hace falta tarjeta gráfica dedicada ni CUDA. Los cuatro modelos (TF-IDF, SVD, SVM y MLP) se entrenan y ejecutan sobre CPU.

Requisitos de red

Una vez instalada, la aplicación no necesita internet. La conexión solo hace falta dos veces: para clonar el repositorio y para instalar las dependencias con `pip install`. A partir de ahí todo es local y los síntomas del paciente no viajan a ningún servidor.

E.3. Instalación

La instalación tiene cuatro pasos: bajar el código, crear un entorno virtual, instalar dependencias y comprobar que los modelos están en su sitio. Los comandos que aparecen a continuación son los de Windows, que es donde se ha desarrollado el prototipo. En Linux y macOS los cambios son mínimos y se indican cuando aparecen.

Descarga del código

El proyecto está en un repositorio público de GitHub. Con Git basta:

```
git clone https://github.com/aos0020/TFG_AOS0020---TEST.git
```

Si no hay Git instalado, se puede bajar el ZIP desde el botón *Code* de la página del repositorio y descomprimirlo en cualquier carpeta del equipo. Para el resto del manual se asume que el código está en una carpeta llamada `TFG_AOS0020--TEST`.

Creación del entorno virtual

Conviene crear un entorno virtual de Python para que las librerías del proyecto no se mezclen con las del sistema. Desde la carpeta del proyecto:

```
python -m venv .venv
.\.venv\Scripts\activate
```

En Linux o macOS la segunda línea se sustituye por `source .venv/bin/activate`. Si el entorno se ha activado bien, la consola muestra el prefijo `(.venv)` delante del prompt.

Instalación de dependencias

Con el entorno activo:

```
pip install -r requirements.txt
```

Esto instala scikit-learn, pandas, numpy, Streamlit, NLTK, joblib y streamlit-mic-recorder. La primera vez puede tardar entre uno y cinco minutos, dependiendo de la conexión.

Configuración de las credenciales de administración

La vista de administración pide usuario y contraseña. El usuario es siempre `admin`; la contraseña la pone cada instalación en su propio equipo. La aplicación la busca en tres sitios, en este orden:

1. En los *secrets* de Streamlit, bajo la clave `admin_password`. Para usar esta vía hay que crear el fichero `.streamlit/secrets.toml` con:

   ```
   admin_password = "la_contraseña_que_se_quiera"
   ```

2. En el fichero `.streamlit/secrets.toml.aos006`, con la misma clave `admin_password`. Esta variante existe para entornos en los que el *secrets* estándar no está disponible.

3. En la variable de entorno `TRIAJE_ADMIN_PASSWORD`.

Si ninguno de los tres está definido, al entrar en la pestaña de administración aparece un aviso y no se permite el acceso. El detalle técnico se explica en el Apéndice D.

Verificación de los modelos entrenados

El repositorio incluye los cuatro modelos ya entrenados como ficheros `.joblib`, uno por carpeta (`TF_IDF/`, `SVD/`, `SVM/` y `RNA/`). En una instalación nueva no hace falta tocarlos. Si por alguna razón se quiere regenerar alguno antes del primer arranque, se ejecuta su script correspondiente, por ejemplo:

```
python TF_IDF\generarModeloTFIDF.py
```

Lo normal, sin embargo, es regenerar los modelos desde la propia aplicación, una vez se han añadido nuevos casos al conjunto de datos.

Arranque de la aplicación

Con todo lo anterior listo, desde la raíz del proyecto:

```
streamlit run app.py
```

Streamlit abre el navegador en `http://localhost:8501`. Si no se abre solo, se puede pegar esa URL manualmente. Para parar la aplicación, se cierra la pestaña y se pulsa `Ctrl + C` en la consola donde estaba corriendo Streamlit.

E.4. Manual del usuario

Al arrancar, la barra lateral muestra un selector *radio* bajo el título *Navegación*, con dos opciones: *Triaje* y *Administración*. La primera es la que ve el usuario final; la segunda, protegida con login, queda para quien mantiene el sistema. La figura **??** muestra el aspecto de la barra lateral con la sesión de administrador abierta.

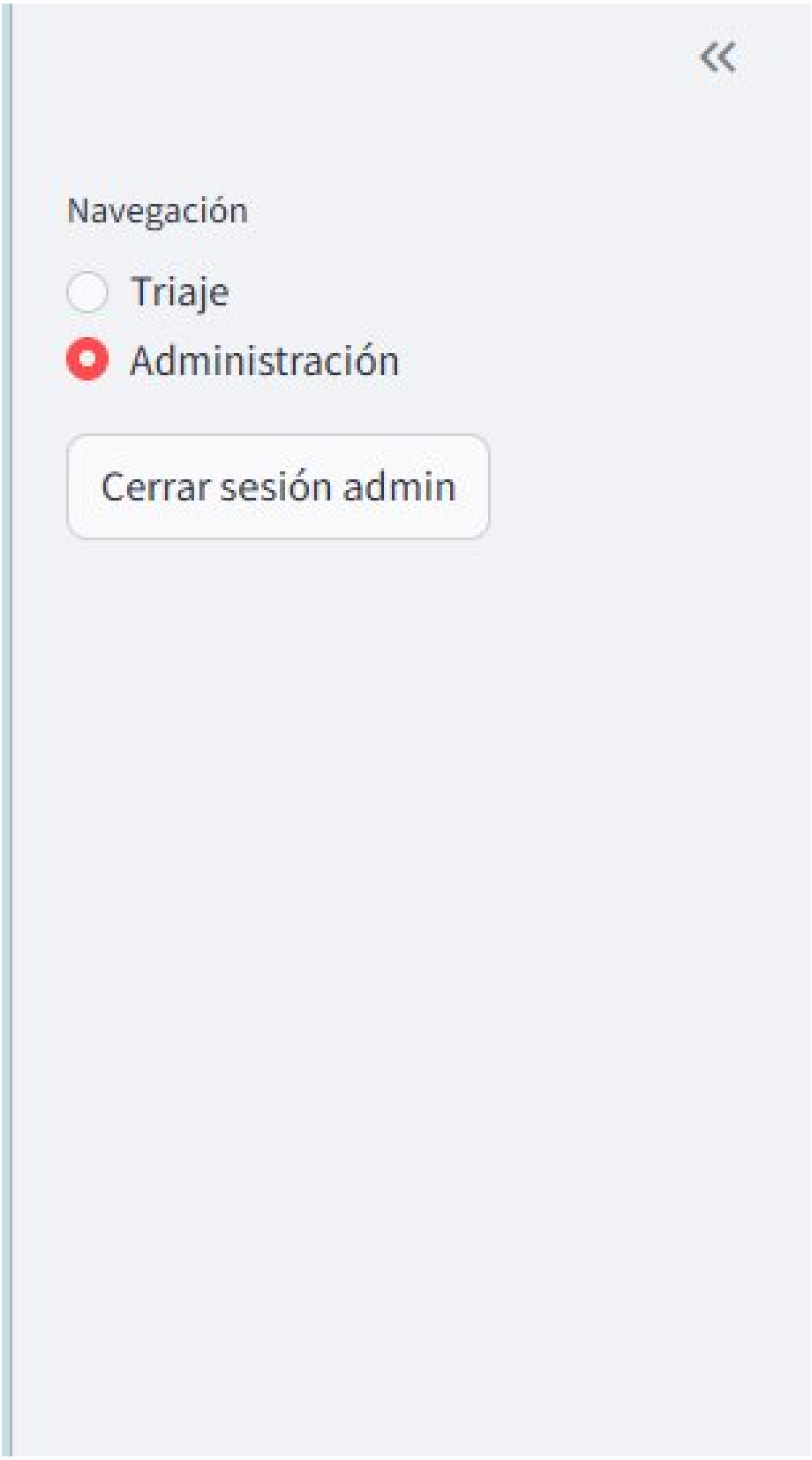

Figura E.1: Barra lateral con la sesión de administrador activa

0.45

Vista del paciente

La pantalla de triaje es la puerta de entrada. Aparece el título *Triaje médico inteligente*, una línea de información con el umbral de confianza y las rondas configuradas, y los controles para introducir los síntomas y elegir cómo se realiza la clasificación.

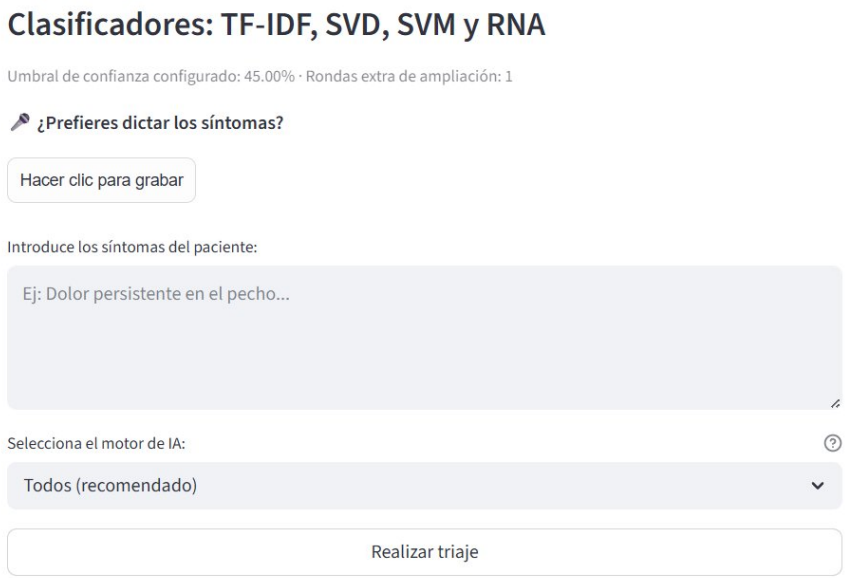

Figura E.2: Pantalla principal del paciente

0.85

Introducción de síntomas

Hay dos formas de meter los síntomas:

- **Por escrito**, en el cuadro *Introduce los síntomas del paciente*. Admite frases tal cual las escribiría el paciente, sin formato especial. Por ejemplo, “me duele mucho la cabeza desde ayer y tengo náuseas”.
- **Por voz**, pulsando el botón *Hacer clic para grabar*. La primera vez el navegador pide permiso para usar el micrófono; después, la grabación se transcribe sola y el texto aparece en el mismo cuadro, donde se puede revisar o corregir antes de seguir.

Las dos vías acaban en el mismo sitio: lo que la aplicación clasifica es siempre el contenido del cuadro de texto. La voz es solo una alternativa al teclado.

Selección del motor de clasificación

El desplegable *Selecciona el motor de IA* tiene cinco opciones: *Todos (recomendado)*, TF-IDF, SVD, SVM y RNA. Por defecto está *Todos (recomendado)*, que ejecuta los cuatro modelos sobre el mismo texto y se queda con el de mayor confianza. Es la opción más razonable para un uso real.

Las otras cuatro fuerzan un modelo concreto. No tienen sentido en un uso normal, pero durante las pruebas o la evaluación del prototipo permiten comparar las respuestas de cada modelo sobre el mismo texto, que es justo lo que interesa para entender sus diferencias.

Lanzamiento del triaje

Con el texto escrito y el modelo elegido, el botón *Realizar triaje* dispara la clasificación. La aplicación limpia el texto, lo vectoriza y lo pasa al clasificador. El resultado aparece en la misma pantalla, y puede caer en uno de tres casos.

Caso 1: confianza suficiente

Cuando la confianza supera el umbral configurado (45 % por defecto), aparece la especialidad sugerida con su porcentaje de confianza y el nombre del modelo que la ha producido. Es lo que ocurre la mayoría de las veces y resuelve el triaje en un solo paso.

Si el motor era *Todos (recomendado)*, debajo del resultado aparece una tabla con lo que ha sugerido cada uno de los cuatro modelos: su especialidad, su confianza y un marcador ($\star$ Elegido) sobre el ganador. Sirve para ver de un vistazo si los modelos están de acuerdo o se contradicen.

Bajo el resultado hay dos botones. *Realizar otro triaje* reinicia el flujo. *Reportar error* se usa cuando el paciente cree que la especialidad sugerida no es la que toca; al pulsarlo, la aplicación pide la especialidad correcta y guarda la corrección en el log para que el administrador la revise. Es la otra vía por la que el sistema aprende: además de los casos en los que el modelo no estaba seguro, también se aprovechan los casos en los que estaba seguro y se equivocó.

Caso 2: confianza insuficiente y ampliación de síntomas

Si el modelo no llega al umbral, la aplicación no abandona. Avisa de que la confianza es baja y pide más información. La figura **??** muestra un ejemplo: el modelo SVD sugiere *Medicina Interna* con un 28,52 % de confianza, por debajo del umbral del 45 %, y el sistema pide una ronda extra.

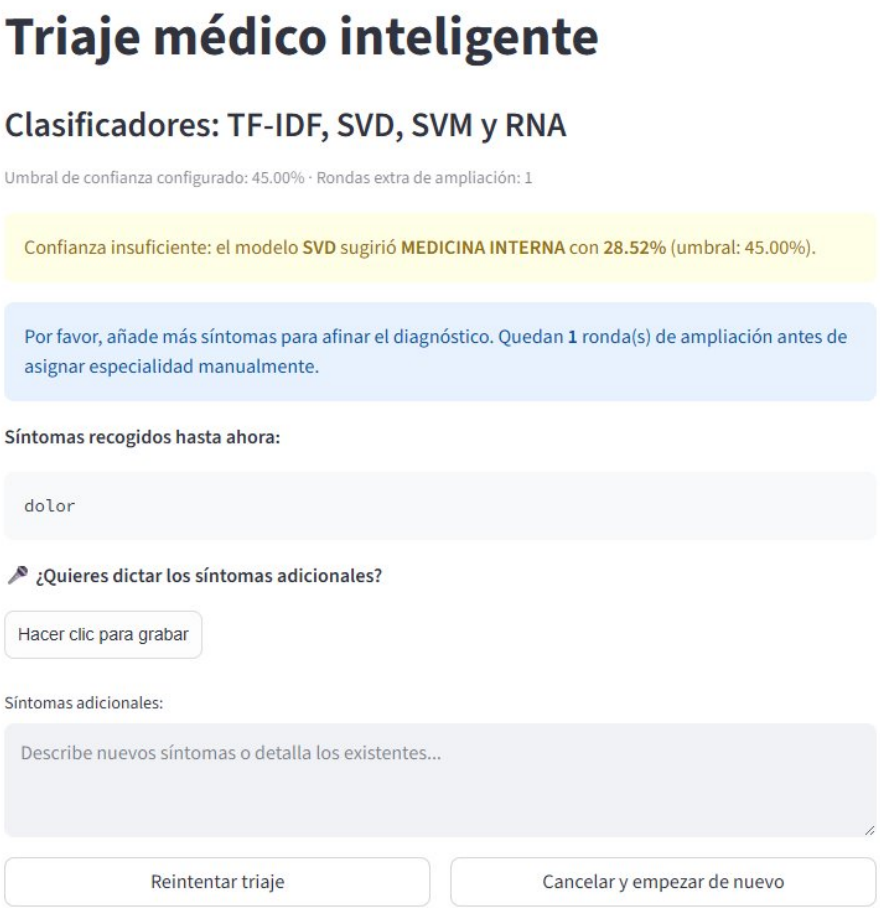

Figura E.3: Ampliación de síntomas tras una predicción de baja confianza

0.85

En esta pantalla, el paciente puede:

- Ver los *síntomas recogidos hasta ahora*, en la parte superior, para repasar lo que ya ha dicho.

- Escribir o dictar más síntomas en el cuadro *Síntomas adicionales*.
- Pulsar *Reintentar triaje* para volver a clasificar con la descripción ampliada.
- Pulsar *Cancelar y empezar de nuevo* si prefiere descartar lo introducido y arrancar desde cero.

Cuántas rondas de ampliación se permiten se configura desde la vista del administrador. Por defecto se hace una sola.

Caso 3: confianza insuficiente tras las rondas de ampliación

Si después de gastar todas las rondas la confianza sigue por debajo del umbral, la aplicación deja que decida el paciente. La figura **??** muestra ese desenlace.

Figura E.4: Selección manual de especialidad tras agotar las rondas

0.85

Aparece un aviso indicando que el modelo no ha llegado a la confianza necesaria, junto con la última sugerencia y su porcentaje. Después se ofrecen dos opciones:

- **Opción 1**: aceptar la sugerencia del sistema, pulsando el botón *Aceptar* seguido del nombre de la especialidad. Es la opción rápida cuando la sugerencia parece razonable.
- **Opción 2**: elegir manualmente otra especialidad desde el desplegable. La opción por defecto es *Medicina General*, que funciona como derivación a atención general cuando no hay nada más claro.

Al pulsar *Confirmar selección*, el caso se guarda en el log para que el administrador lo revise después. Las consultas que el modelo no ha sabido resolver con seguridad son justo las que más interesa revisar y reincorporar al entrenamiento.

Vista del administrador

Antes de entrar, la aplicación pide usuario y contraseña en un formulario. El usuario es `admin`; la contraseña es la que se haya configurado durante la instalación. Mientras la sesión esté abierta, la barra lateral muestra un botón *Cerrar sesión admin* para salir de la zona privada.

Dentro, la página se titula *Administración de datos* y agrupa cuatro bloques, en este orden: configuración del triaje, alta de nuevos registros, regeneración de los modelos y registro de triajes con baja confianza o reportes de error.

Configuración del triaje

Aquí se ajustan los dos parámetros que controlan cómo responde el sistema ante casos dudosos. La figura **??** muestra los controles.

Figura E.5: Configuración del umbral de confianza y de las rondas de ampliación

0.7

Los parámetros son:

- **Umbral mínimo de confianza (%)**, un deslizador de 0 a 100. Es el porcentaje mínimo que debe alcanzar el modelo para que su predicción se acepte sin pedir más información. Cuanto más alto, más prudente es el sistema, pero también pasa por la ampliación más casos. Cuanto más bajo, más resolutivo, pero también más arriesgado.
- **Rondas extra de ampliación de síntomas**, un contador entre 0 y 10 que indica cuántas veces se pedirá al paciente que añada información antes de pasar a selección manual. Por defecto, una sola ronda.

Al pulsar *Guardar configuración* los nuevos valores se escriben en `config.json`, en la raíz del proyecto, y se aplican a todas las consultas siguientes.

Añadir síntomas y especialidad

Esta sección sirve para añadir manualmente pares de síntomas y especialidad. Es útil cuando se quieren incorporar casos sacados de fuentes externas (un manual clínico, un protocolo asistencial) sin tener que pasar por la vista del paciente. La figura **??** muestra el formulario.

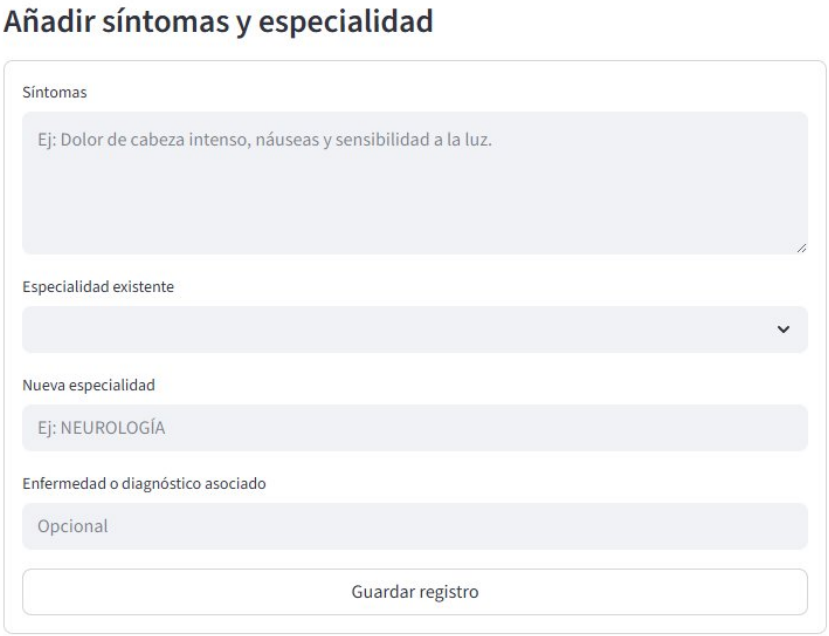

Figura E.6: Formulario para añadir nuevos pares de síntomas y especialidad

0.7

Los campos son:

- **Síntomas**: un cuadro de texto multilínea para describir el caso.
- **Especialidad existente**: un desplegable con las especialidades que ya están en el dataset.
- **Nueva especialidad**: un campo libre que solo se usa si la especialidad no aparece todavía en la lista anterior.
- **Enfermedad o diagnóstico asociado**: opcional, para dejar constancia del diagnóstico concreto cuando se conoce.

Al pulsar *Guardar registro*, el caso se añade al fichero principal `Datos/Dataset.5000.Registros.Ma` con el siguiente `record_id` libre. La aplicación devuelve un mensaje confirmando el identificador asignado.

Regenerar modelos

Cuando ya hay casos nuevos suficientes, el administrador puede reentrenar los modelos sin salir de la interfaz. La figura **??** muestra los controles. La regeneración combina el dataset principal con los casos revisados de `Datos/triajes_baja_confianza_revisados.csv` y sobrescribe los `.joblib` de cada carpeta de modelo.

Figura E.7: Botones para regenerar todos los modelos o uno solo

0.7

Hay dos formas de hacerlo:

- **Regenerar todos los modelos**: el botón rojo lanza los cuatro scripts uno detrás de otro (TF-IDF, SVD, SVM y RNA). Una barra de progreso indica en qué modelo va y, al terminar, aparece un resumen con el resultado de cada uno.
- **Regenerar individualmente**: cuatro botones separados, uno por modelo. Útil cuando solo interesa actualizar uno concreto, por ejemplo durante las pruebas.

El reentrenamiento del MLP es el más lento y puede llevar varios minutos según el equipo. Mientras dura, la aplicación sigue siendo usable, pero los modelos nuevos no entran en juego hasta que el proceso acaba.

Registro de triajes (baja confianza y reportes de error)

En la parte inferior de la página se muestran los casos guardados en `logs/triaje_fallback_log.csv`. Allí se acumulan dos tipos de incidencias:

triajes que no alcanzaron el umbral tras agotar las rondas, y reportes de error generados por el paciente desde la pantalla de resultado. La figura **??** muestra el aspecto de la pantalla.

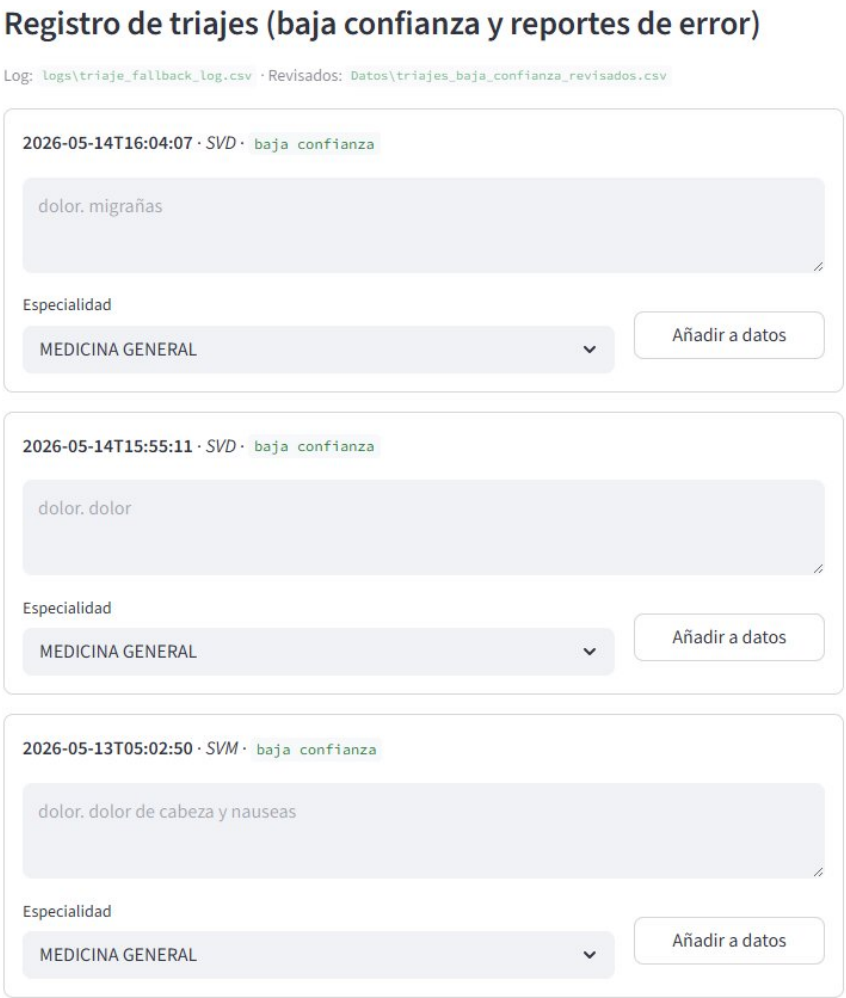

Figura E.8: Revisión y reasignación manual de casos de baja confianza o reportados como erróneos

0.85

Para cada caso, se muestra la fecha y hora, el modelo que clasificó, el motivo de la incidencia (*baja confianza* o *reporte de error*) y el texto completo de los síntomas. Desde ahí, el administrador puede:

- Elegir la especialidad correcta en el desplegable *Especialidad*, que parte con la que el paciente había marcado en su momento.
- Pulsar *Añadir a datos* para volcar el caso etiquetado a `Datos/triajes_baja_confianza`

Los casos que el clasificador no resolvió bien (porque la confianza fue baja o porque el paciente discrepó) se convierten en ejemplos nuevos para el próximo entrenamiento. Las especialidades poco representadas en el dataset original van ganando ejemplos a medida que se revisan, lo que compensa poco a poco el desequilibrio inicial entre clases.

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo incluirá una reflexión personal del alumnado sobre los aspectos de la sostenibilidad que se abordan en el trabajo. Se pueden incluir tantas subsecciones como sean necesarias con la intención de explicar las competencias de sostenibilidad adquiridas durante el alumnado y aplicadas al Trabajo de Fin de Grado.

Más información en el documento de la CRUE https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sosteniblidad_Crue2012.pdf.

Este anexo tendrá una extensión comprendida entre 600 y 800 palabras.

Siglas

MLP *Multi-layer Perceptron*. 1, 6, 7, 11, 14, 50, 59, 62, 74

PLN Procesamiento de Lenguaje Natural. 4, 6, 12, 13

RNA Red Neuronal Artificial. 1, 7, 11, 14, 19, 32, 68, 74

SVD *Singular Value Decomposition*. 1, 5, 6, 11, 12, 13, 14, 19, 32, 62, 68, 69, 74

SVM *Support Vector Machine*. 1, 5, 6, 11, 12, 14, 19, 32, 54, 59, 62, 68, 74

TF-IDF *Term Frequency – Inverse Document Frequency*. 1, 4, 5, 6, 11, 12, 13, 14, 19, 32, 50, 62, 68, 74

Glosario

early stopping Técnica de regularización en el entrenamiento de redes neuronales. Consiste en detener el entrenamiento cuando el error de validación deja de mejorar durante un número determinado de épocas, aunque el error de entrenamiento siga bajando. Evita que el modelo memorice los datos de entrenamiento y pierda capacidad de generalizar. 14

fallback Flujo alternativo que se ejecuta cuando la predicción del modelo no alcanza la confianza mínima exigida. El sistema pide más síntomas durante un número configurable de rondas y, agotadas estas, deja que el paciente acepte la sugerencia del modelo o elija manualmente otra especialidad. 12, 15

GridSearchCV Búsqueda de rejilla con validación cruzada (*Grid Search Cross-Validation*). Procedimiento de exploración estructurada sobre un conjunto discreto de hiperparámetros para encontrar la combinación óptima. 54, 57

standalone Sistema o aplicación que funciona de forma autónoma, sin depender de servicios externos, conexión a internet ni infraestructura adicional. 9, 11, 16, 36

Stemming Proceso lingüístico de reducción de una palabra a su raíz o tallo (*stem*). Técnica de bajo coste computacional comparada con la lematización. 13, 54

TruncatedSVD Variante de la Descomposición en Valores Singulares que opera directamente sobre matrices dispersas sin centrarlas, utilizada

en NLP para reducción de dimensionalidad sobre matrices TF-IDF. 4, 5, 13, 50

Bibliografía

[1] Agencia Tributaria. Retenciones sobre rendimientos del trabajo. tablas de retención IRPF, 2026. [Internet; consultado mayo-2026].

[2] R. Bapt. streamlit-mic-recorder: Streamlit component for audio recording and speech-to-text. https://pypi.org/project/streamlit-mic-recorder/, 2024. [Internet; consultado 13-mayo-2026].

[3] BOE. XVIII Convenio Colectivo Estatal de Empresas de Consultoría, Tecnologías de la Información y Estudios de Mercado y de la Opinión Pública, 2023. [Internet; consultado mayo-2026].

[4] GitHub, Inc. Github: Let's build from here. https://github.com/, 2026. [Internet; consultado 13-mayo-2026].

[5] scikit-learn developers. scikit-learn: Machine learning in Python, 2026. [Internet; consultado mayo-2026].

[6] Seguridad Social. Bases y tipos de cotización 2026, 2026. [Internet; consultado mayo-2026].

[7] Snowflake Inc. Streamlit: A faster way to build and share data apps. https://streamlit.io/, 2026. [Internet; consultado 13-mayo-2026].

[8] Streamlit Inc. Streamlit documentation, 2026. [Internet; consultado mayo-2026].

[9] Zube. Zube — agile project management for GitHub, 2026. [Internet; consultado mayo-2026].